

C++ Standard Library Immediate Issues to be moved in Issaquah, Feb. 2023

Doc. no. P2790R0

Date: 2023-02-10

Audience: WG21

Reply to: Jonathan Wakely <lwgchair@gmail.com>

Immediate Issues

[3441](#). Misleading note about calls to customization points

Section: 16.4.5.2.1 [\[namespace.std\]](#) **Status:** [Immediate](#) **Submitter:** Michael Park **Opened:** 2020-05-08 **Last modified:** 2023-02-10

Priority: 1

View other [active issues](#) in [\[namespace.std\]](#).

View all other [issues](#) in [\[namespace.std\]](#).

Discussion:

[P0551](#) (Thou Shalt Not Specialize std Function Templates!) added a clause in [\[namespace.std\]](#)/7:

Other than in namespace std or in a namespace within namespace std, a program may provide an overload for any library function template designated as a customization point, provided that (a) the overload's declaration depends on at least one user-defined type and (b) the overload meets the standard library requirements for the customization point. (footnote 174) *[Note: This permits a (qualified or unqualified) call to the customization point to invoke the most appropriate overload for the given arguments. — end note]*

Given that `std::swap` is a designated customization point, the note seems to suggest the following:

```
namespace N {
    struct X {};
    void swap(X&, X&) {}
}

N::X a, b;
std::swap(a, b); // qualified call to customization point finds N::swap?
```

This is not what happens, as the call to `std::swap` does not find `N::swap`.

[2020-07-17; Priority set to 1 in telecon]

Related to [3442](#).

[2020-09-11; discussed during telecon]

The note is simply bogus, not backed up by anything normative. The normative part of the paragraph is an unacceptable landgrab on those identifiers. We have no right telling users they can't use the names `data` and `size` unless they do exactly what we say `std::data` and `std::size` do. The library only ever uses `swap` unqualified, so the effect of declaring the others as designated customization points is unclear.

The rule only needs to apply to such overloads *when actually found by overload resolution* in a context that expects the semantics of the customization point.

Frank: do we need to designate `operator<<` as a customization point? Users overload that in their own namespaces all the time.

Walter: This clearly needs a paper.

[2020-10-02; status to Open]

Previous resolution [SUPERSEDED]:

This wording is relative to [N4861](#).

1. Modify 16.4.5.2.1 [\[namespace.std\]](#) as indicated:

-7- Other than in namespace std or in a namespace within namespace std, a program may provide an overload for any library function template designated as a customization point, provided that (a) the overload's declaration depends on at least one user-defined type and (b) the overload meets the standard library requirements for the customization point. (footnote 173) *[Note: This permits a (qualified or unqualified) call to the customization point to invoke the most appropriate overload for the given arguments. — end note]*

[Issaquah 2023-02-09; Jonathan provides improved wording]

The normative part of 16.4.5.2.1 [\[namespace.std\]](#) paragraph 7 (i.e. not the note and the footnote) does not actually do anything except tell users they're not allowed to use the names `begin`, `size` etc. unless they conform to the provisions of paragraph 7. This means a program that contains the following declaration might have

undefined behaviour:

```
namespace user { int data(int, int); }
```

This is not OK! It's not even clear what "the requirements for the customization point" are, if users wanted to meet them. It's not even clear what "provide an overload" means.

In particular, paragraph 7 does not give permission for the designated customization points to actually *use* such overloads. Just by forbidding users from using `data` for their own functions isn't sufficient to allow the library to use ADL to call `data`, and it's unclear why we'd want that anyway (what problem with `std::data` are we trying to solve that way?). As shown in LWG 3442, if `std::data` became a customization point that would be a backwards-incompatible change, and create a portability problem if some implementations did that and others didn't.

So the non-normative note and footnote make claims that do not follow from the normative wording, and should be removed.

The only one of the designated customization points that is actually looked up using ADL is `std::swap`, but how that works is fully specified by 16.4.2.2 [contents] and 16.4.4.3 [swappable.requirements]. The additional specification that it's a designated customization point serves no benefit. In particular, the permission that the note and footnote claim exists is not needed. We specify precisely how `swap` calls are performed. We don't even need to say that user overloads of `swap` in their own namespaces must meet the library requirements, because the "swappable with" and *Cpp17Swappable* requirements state the required semantics, and functions that use `swap` have preconditions that the types are swappable. So we correctly impose preconditions **in the places that actually call `swap`**, and don't need to globally make it undefined for any function called `swap` to not meet the requirements, even if that function is never found by ADL by the library (e.g. because it's in a namespace that is never an associated namespace of any types used with library components that require swappable types).

Paragraph 7 and its accompanying notes should go.

[Issaquah 2023-02-09; LWG]

Move to Immediate for C++23

Proposed resolution:

This wording is relative to [N4928](#).

1. Modify 16.4.5.2.1 [namespace.std] as indicated:

[Drafting note: This should also remove the only index entry for "customization point".]

~~7- Other than in namespace std or in a namespace within namespace std, a program may provide an overload for any library function template designated as a customization point, provided that (a) the overload's declaration depends on at least one user-defined type and (b) the overload meets the standard library requirements for the customization point.~~¹⁶³

~~[Note 3: This permits a (qualified or unqualified) call to the customization point to invoke the most appropriate overload for the given arguments. — end note]~~

~~163) Any library customization point must be prepared to work adequately with any user-defined overload that meets the minimum requirements of this document. Therefore an implementation can elect, under the as-if rule (6.9.1 [intro.execution]), to provide any customization point in the form of an instantiated function object (22.10 [function.objects]) even though the customization point's specification is in the form of a function template. The template parameters of each such function object and the function parameters and return type of the object's operator(.) must match those of the corresponding customization point's specification.~~

2. Modify 22.2.2 [utility.swap] as indicated:

```
template<class T>
constexpr void swap(T& a, T& b) noexcept(see below);
```

- 1- Constraints: `is_move_constructible_v<T>` is true and `is_move_assignable_v<T>` is true.
- 2- Preconditions: Type `T` meets the *Cpp17MoveConstructible* (Table 32) and *Cpp17MoveAssignable* (Table 34) requirements.
- 3- Effects: Exchanges values stored in two locations.
- 4- Remarks: ~~This function is a designated customization point (16.4.5.2.1 [namespace.std]).~~ The exception specification is equivalent to:

```
is_nothrow_move_constructible_v<T> && is_nothrow_move_assignable_v<T>
```

3. Modify 25.7 [iterator.range] as indicated:

- 1- In addition to being available via inclusion of the `<iterator>` header, the function templates in 25.7 [iterator.range] are available when any of the following headers are included: `<array>` (24.3.2 [array.syn]), `<deque>` (24.3.3 [deque.syn]), `<forward_list>` (24.3.4 [forward.list.syn]), `<list>` (24.3.5 [list.syn]), `<map>` (24.4.2 [associative.map.syn]), `<regex>` (32.3 [re.syn]), `<set>` (24.4.3 [associative.set.syn]), `<span>` (24.7.2.1 [span.syn]), `<string>` (23.4.2 [string.syn]), `<string_view>` ([string.view.syn]), `<unordered_map>` (24.5.2 [unord.map.syn]), `<unordered_set>` (24.5.3 [unord.set.syn]), and `<vector>` (24.3.6 [vector.syn]). ~~Each of these templates is a designated customization point (16.4.5.2.1 [namespace.std]).~~

3622. Misspecified transitivity of equivalence in §[unord.req.general]

Section: 24.2.8.1 [unord.req.general] Status: [Immediate](#) Submitter: Thomas Köppe Opened: 2021-10-20 Last modified: 2023-02-09

Priority: 2

Discussion:

The paper [P2077R3](#) ("Heterogeneous erasure overloads for associative containers") adds a new variable `kx` with specific meaning for use in the Table of Unordered Associative Container Requirements, 24.2.8.1 [\[unord.req.general\]](#) p11, which is meant to stand for an equivalence class of heterogeneous values that can be compared with container keys.

One property required of `kx` is transitivity of equality/equivalence, but this is currently specified as:

"`kx` is a value such that [...] $(eq(r1, kx) \ \&\& \ eq(r1, r2)) == eq(r2, kx)$ [...], where `r1` and `r2` are [any] keys".

But this doesn't seem right. Transitivity means that $eq(r1, kx) \ \&\& \ eq(r1, r2)$ being true implies $eq(r2, kx)$ being true, but it does *not* imply that both sides are equal in general. In particular, $eq(r2, kx)$ can be true even when $eq(r1, kx) \ \&\& \ eq(r1, r2)$ is false.

More abstractly, equality is transitive, but inequality is not.

The new wording appears to have been copied from the pre-existing wording for the variable "`ke`", which suffers from the same problem, and so we propose to fix both cases.

[2022-01-29; Reflector poll]

Set priority to 2 after reflector poll.

Previous resolution [SUPERSEDED]:

This wording is relative to [N4901](#).

1. Modify 24.2.8.1 [\[unord.req.general\]](#) as indicated:

[...]

(11.19) — `ke` is a value such that

(11.19.1) — $eq(r1, ke) == eq(ke, r1)$,

(11.19.2) — $hf(r1) == hf(ke)$ if $eq(r1, ke)$ is true, and

(11.19.3) — ~~$(eq(r1, ke) \ \&\& \ eq(r1, r2)) == eq(r2, ke)$~~ $eq(ke, r2)$ is true if $eq(ke, r1) \ \&\& \ eq(r1, r2)$ is true.

where `r1` and `r2` are keys of elements in `a_tran`,

(11.20) — `kx` is a value such that

(11.20.1) — $eq(r1, kx) == eq(kx, r1)$,

(11.20.2) — $hf(r1) == hf(kx)$ if $eq(r1, kx)$ is true,

(11.20.3) — ~~$(eq(r1, kx) \ \&\& \ eq(r1, r2)) == eq(r2, kx)$~~ $eq(kx, r2)$ is true if $eq(kx, r1) \ \&\& \ eq(r1, r2)$ is true, and

(11.20.4) — `kx` is not convertible to either `iterator` or `const_iterator`,

where `r1` and `r2` are keys of elements in `a_tran`,

[...]

[2022-02-07 Tim comments and provides updated wording]

For heterogeneous lookup on unordered containers to work properly, we need all keys comparing equal to the transparent key to be grouped together. Since the only keys guaranteed to be so grouped are the ones that are equal according to `eq`, we cannot allow $eq(r1, r2) == false$ but $eq(r1, ke) == true \ \&\& \ eq(r2, ke) == true$. The one-way transitivity of equality is insufficient.

We need both of the following:

- if $eq(r1, ke)$ is true and $eq(r1, r2)$ is true then $eq(r2, ke)$ is true.
- if $eq(r1, ke)$ is true and $eq(r2, ke)$ is true then $eq(r1, r2)$ is true

In a table:

<code>eq(r1, ke)</code>	<code>eq(r1, r2)</code>	<code>eq(r2, ke)</code>	OK?
T	T	T	Y
T	T	F	N
T	F	T	N
T	F	F	Y
F	T	T	N
F	T	F	Y
F	F	T	Y
F	F	F	Y

[Issaquah 2023-02-08; LWG]

Proposed resolution:

This wording is relative to [N4928](#).

1. Modify 24.2.8.1 [\[unord.req.general\]](#) as indicated:

[...]

(10.20) — ke is a value such that

(10.20.1) — $\text{eq}(r1, \text{ke}) == \text{eq}(\text{ke}, r1)$,

(10.20.2) — $\text{hf}(r1) == \text{hf}(\text{ke})$ if $\text{eq}(r1, \text{ke})$ is true, and

(10.20.3) — ~~$(\text{eq}(r1, \text{ke}) \ \&\& \ \text{eq}(r1, r2)) == \text{eq}(r2, \text{ke})$~~ if any two of $\text{eq}(r1, \text{ke})$, $\text{eq}(r2, \text{ke})$ and $\text{eq}(r1, r2)$ are true, then all three are true.

where r1 and r2 are keys of elements in a_tran,

(10.21) — kx is a value such that

(10.21.1) — $\text{eq}(r1, \text{kx}) == \text{eq}(\text{kx}, r1)$,

(10.21.2) — $\text{hf}(r1) == \text{hf}(\text{kx})$ if $\text{eq}(r1, \text{kx})$ is true,

(10.21.3) — ~~$(\text{eq}(r1, \text{kx}) \ \&\& \ \text{eq}(r1, r2)) == \text{eq}(r2, \text{kx})$~~ if any two of $\text{eq}(r1, \text{kx})$, $\text{eq}(r2, \text{kx})$ and $\text{eq}(r1, r2)$ are true, then all three are true, and

(10.21.4) — kx is not convertible to either iterator or const_iterator,

where r1 and r2 are keys of elements in a_tran,

[...]

[3631](#). `basic_format_arg(T&&)` should use `remove_cvref_t<T>` throughout

Section: 22.14.8.1 [\[format.arg\]](#) Status: [Immediate](#) Submitter: Tim Song Opened: 2021-11-03 Last modified: 2023-02-09

Priority: 3

View other [active issues](#) in [\[format.arg\]](#).

View all other [issues](#) in [\[format.arg\]](#).

Discussion:

[P2418R2](#) changed `basic_format_arg`'s constructor to take a forwarding reference but didn't change 22.14.8.1 [\[format.arg\]](#)/5 which inspects various properties of T. Now that the deduced type can be *cvref*-qualified, they need to be removed before the checks.

[2022-01-29; Reflector poll]

Set priority to 3 after reflector poll.

Two suggestions to just change it to be `T&` because we don't need forwarding references here, and only accepting lvalues prevents forming dangling references.

Previous resolution [SUPERSEDED]:

This wording is relative to [N4901](#).

1. Modify 22.14.8.1 [\[format.arg\]](#) as indicated:

```
template<class T> explicit basic_format_arg(T&& v) noexcept;
```

~~-?- Let TD be `remove_cvref_t<T>`.~~

-4- Constraints: The template specialization

```
typename Context::template formatter_type<remove_cvref_t<T>TD>
```

meets the *BasicFormatter* requirements (22.14.6.1 [\[formatter.requirements\]](#)). The extent to which an implementation determines that the specialization meets the *BasicFormatter* requirements is unspecified, except that as a minimum the expression

```
typename Context::template formatter_type<remove_cvref_t<T>TD>()  
.format(declval<T>(), declval<Context&>())
```

shall be well-formed when treated as an unevaluated operand (7.2.3 [\[expr.context\]](#)).

-5- Effects:

- (5.1) — if `TD` is `bool` or `char_type`, initializes value with `v`;
- (5.2) — otherwise, if `TD` is `char` and `char_type` is `wchar_t`, initializes value with `static_cast<wchar_t>(v)`;
- (5.3) — otherwise, if `TD` is a signed integer type (6.8.2 [basic.fundamental]) and `sizeof(TD) <= sizeof(int)`, initializes value with `static_cast<int>(v)`;
- (5.4) — otherwise, if `TD` is an unsigned integer type and `sizeof(TD) <= sizeof(unsigned int)`, initializes value with `static_cast<unsigned int>(v)`;
- (5.5) — otherwise, if `TD` is a signed integer type and `sizeof(TD) <= sizeof(long long int)`, initializes value with `static_cast<long long int>(v)`;
- (5.6) — otherwise, if `TD` is an unsigned integer type and `sizeof(TD) <= sizeof(unsigned long long int)`, initializes value with `static_cast<unsigned long long int>(v)`;
- (5.7) — otherwise, initializes value with `handle(v)`.

[2022-10-19; Jonathan provides improved wording]

During reflector prioritization it was pointed out that forwarding references are unnecessary (arguments are always lvalues), and so using `T&` would be simpler. In order to resolve the overload ambiguities discussed in 3718 replace all unary constructor overloads with a single constructor that works for any formattable type.

Previous resolution [SUPERSEDED]:

This wording is relative to N4917.

1. Modify 22.14.8.1 [format.arg] as indicated:

```
template<class T> explicit basic_format_arg(T&& v) noexcept;
```

-4- Constraints: `T` satisfies `formattable<char_type>`. The template specialization

```
typename Context::template formatter_type<remove_cvref_t<T>>
```

~~meets the `BasicFormatter` requirements (22.14.6.1 [formatter.requirements]). The extent to which an implementation determines that the specialization meets the `BasicFormatter` requirements is unspecified, except that as a minimum the expression~~

```
typename Context::template formatter_type<remove_cvref_t<T>>()
    .format(declval<T&>(), declval<Context&>())
```

~~shall be well-formed when treated as an unevaluated operand (7.2.3 [expr.context]).~~

-?- Preconditions: If decay `t<T>` is `char type*` or `const char type*`, `static_cast<const char type*>(v)` points to a NTCTS (3.36 [defs.ntcts]).

-5- Effects: Let `TD` be `remove_const t<T>`.

- (5.1) — if `TD` is `bool` or `char_type`, initializes value with `v`;
- (5.2) — otherwise, if `TD` is `char` and `char_type` is `wchar_t`, initializes value with `static_cast<wchar_t>(v)`;
- (5.3) — otherwise, if `TD` is a signed integer type (6.8.2 [basic.fundamental]) and `sizeof(TD) <= sizeof(int)`, initializes value with `static_cast<int>(v)`;
- (5.4) — otherwise, if `TD` is an unsigned integer type and `sizeof(TD) <= sizeof(unsigned int)`, initializes value with `static_cast<unsigned int>(v)`;
- (5.5) — otherwise, if `TD` is a signed integer type and `sizeof(TD) <= sizeof(long long int)`, initializes value with `static_cast<long long int>(v)`;
- (5.6) — otherwise, if `TD` is an unsigned integer type and `sizeof(TD) <= sizeof(unsigned long long int)`, initializes value with `static_cast<unsigned long long int>(v)`;
- (5.?) — otherwise, if `TD` is a standard floating-point type, initializes value with `v`;
- (5.?) — otherwise, if `TD` is a specialization of `basic_string_view` or `basic_string` and `TD::value_type` is `char_type`, initializes value with `basic_string_view<char_type>(v.data(), v.size())`;
- (5.?) — otherwise, if decay `t<TD>` is `char type*` or `const char type*`, initializes value with `static_cast<const char type*>(v)`;
- (5.?) — otherwise, if `is_void v<remove_pointer t<TD>>` is true or `is_null_pointer v<TD>` is true, initializes value with `static_cast<const void*>(v)`;
- (5.7) — otherwise, initializes value with `handle(v)`.

~~-?- [Note: Constructing basic_format_arg from a pointer to a member is ill-formed unless the user provides an enabled specialization of formatter for that pointer to member type. — end note]~~

```
explicit basic_format_arg(float n) noexcept;  
explicit basic_format_arg(double n) noexcept;  
explicit basic_format_arg(long double n) noexcept;
```

~~-6- Effects: Initializes value with n.~~

```
explicit basic_format_arg(const char_type* s) noexcept;
```

~~-7- Preconditions: s points to a NTCTS (3.36 [defs.ntcts]).~~

~~-8- Effects: Initializes value with s.~~

```
template<class traits>  
explicit basic_format_arg(basic_string_view<char_type, traits> s) noexcept;
```

~~-9- Effects: Initializes value with basic_string_view<char_type>(s.data(), s.size()).~~

```
template<class traits, class Allocator>  
explicit basic_format_arg(  
const basic_string<char_type, traits, Allocator>& s) noexcept;
```

~~-10- Effects: Initializes value with basic_string_view<char_type>(s.data(), s.size()).~~

```
explicit basic_format_arg(nullptr_t) noexcept;
```

~~-11- Effects: Initializes value with static_cast<const void*>(nullptr).~~

```
template<class T> explicit basic_format_arg(T* p) noexcept;
```

~~-12- Constraints: is_void_v<T> is true.~~

~~-13- Effects: Initializes value with p.~~

~~-14- [Note: Constructing basic_format_arg from a pointer to a member is ill-formed unless the user provides an enabled specialization of formatter for that pointer to member type. — end note]~~

2. Modify 22.14.8.1 [format.arg] p17 and p18 as indicated:

```
template<class T> explicit handle(T& v) noexcept;
```

-17- Let

— (17.1) TD be remove_cvref<const T>T>,

~~— (17.2) const_formattable be true if typename Context::template formatter_type<TD> f().format(declval<const TD>(), declval<Context>()) is well-formed, otherwise false;~~

— (17.3) TQ be const TD if ~~const_formattable is true~~ **const TD satisfies formattable<char_type>** and TD otherwise.

-18- Mandates: ~~const_formattable~~ **formattable<const TD, char_type>** **||**
~~!is_const_v<remove_reference_t<T>~~ is true.

-19- Effects: Initializes ptr_ with addressof(val) and format_ with

```
{(basic_format_parse_context<char_type>& parse_ctx,  
Context& format_ctx, const void* ptr) {  
    typename Context::template formatter_type<TD> f;  
    parse_ctx.advance_to(f.parse(parse_ctx));  
    format_ctx.advance_to(f.format(*const_cast<TQ*>(static_cast<const TD*>(ptr)),  
                                format_ctx));  
}
```

[2022-11-10; Jonathan provides improved wording]

[2022-11-29; Casey expands the issue to also cover make_format_args]

[2023-01-11; Jonathan adds the missing edits to the class synopsis]

[Issaquah 2023-02-07; LWG]

Edited proposed resolution to remove extra **||** in concept definition and capitalize start of (5.1). Move to Immediate for C++23

Proposed resolution:

This wording is relative to [N4917](#).

1. Modify 22.14.6.2 [\[format.formattable\]](#) as indicated:

-1- Let `fmt-iter-for<charT>` be an unspecified type that models `output_iterator<const charT>` (25.3.4.10 [\[iterator.concept.output\]](#)).

```
template<class T, class Context,
        class Formatter = typename Context::template formatter_type<remove_const_t<T>>>
concept formattable-with = // exposition only
    semiregular<Formatter> &&
    requires (Formatter& f, const Formatter& cf, T&& t, Context fc,
              basic_format_parse_context<typename Context::char_type> pc)
    {
        { f.parse(pc) } -> same as<typename decltype(pc)::iterator>;
        { cf.format(t, fc) } -> same as<typename Context::iterator>;
    };

template<class T, class charT>
concept formattable =
    semiregular<formatter<remove_cvref_t<T>, charT>> &&
    requires (formatter<remove_cvref_t<T>, charT> f,
              const formatter<remove_cvref_t<T>, charT> cf,
              T t,
              basic_format_context<fmt_iter_for<charT>, charT> fc,
              basic_format_parse_context<charT> pc) {
        { f.parse(pc) } -> same as<basic_format_parse_context<charT>::iterator>;
        { cf.format(t, fc) } -> same as<fmt_iter_for<charT>>;
    };
formattable-with<remove_reference_t<T>, basic_format_context<fmt_iter_for<charT>>>;
```

2. Modify 22.14.8.1 [\[format.arg\]](#) as indicated:

```
namespace std {
template<class Context>
class basic_format_arg {
public:
    class handle;

private:
    using char_type = typename Context::char_type; // exposition only

    variant<monostate, bool, char_type,
            int, unsigned int, long long int, unsigned long long int,
            float, double, long double,
            const char_type*, basic_string_view<char_type>,
            const void*, handle> value; // exposition only

    template<class T> explicit basic_format_arg(T&& v) noexcept; // exposition only
    explicit basic_format_arg(float n) noexcept; // exposition only
    explicit basic_format_arg(double n) noexcept; // exposition only
    explicit basic_format_arg(long double n) noexcept; // exposition only
    explicit basic_format_arg(const char_type* s); // exposition only

    template<class traits>
    explicit basic_format_arg(
        basic_string_view<char_type, traits> s) noexcept; // exposition only

    template<class traits, class Allocator>
    explicit basic_format_arg(
        const basic_string<char_type, traits, Allocator>& s) noexcept; // exposition only

    explicit basic_format_arg(nullptr_t) noexcept; // exposition only

    template<class T>
    explicit basic_format_arg(T* p) noexcept; // exposition only
...
template<class T> explicit basic_format_arg(T&& v) noexcept;
```

-4- Constraints: `T` satisfies `formattable-with<Context>`. The template specialization

```
typename Context::template formatter_type<remove_cvref_t<T>>
```

meets the *BasicFormatter* requirements (22.14.6.1 [\[formatter.requirements\]](#)). The extent to which an implementation determines that the specialization meets the *BasicFormatter* requirements is unspecified, except that as a minimum the expression

```
typename Context::template formatter_type<remove_cvref_t<T>>()
    .format(decval<T>(), decval<Context>())
```

shall be well-formed when treated as an unevaluated operand (7.2.3 [\[expr.context\]](#)).

-?- **Preconditions:** If `decay_t<T>` is `char_type*` or `const char_type*`, `static cast<const char_type*>(v)` points to a NTCTS (3.36 [\[defs.ntcts\]](#)).

-5- **Effects:** Let `TD` be `remove_const_t<T>`.

(5.1) — If `TD` is `bool` or `char_type`, initializes value with `v`;

(5.2) — otherwise, if `TD` is `char` and `char_type` is `wchar_t`, initializes value with `static_cast<wchar_t>(v)`;

(5.3) — otherwise, if `TD` is a signed integer type (6.8.2 [basic.fundamental]) and `sizeof(TD) <= sizeof(int)`, initializes value with `static_cast<int>(v)`;

(5.4) — otherwise, if `TD` is an unsigned integer type and `sizeof(TD) <= sizeof(unsigned int)`, initializes value with `static_cast<unsigned int>(v)`;

(5.5) — otherwise, if `TD` is a signed integer type and `sizeof(TD) <= sizeof(long long int)`, initializes value with `static_cast<long long int>(v)`;

(5.6) — otherwise, if `TD` is an unsigned integer type and `sizeof(TD) <= sizeof(unsigned long long int)`, initializes value with `static_cast<unsigned long long int>(v)`;

(5.7) — otherwise, if `TD` is a standard floating-point type, initializes value with `v`;

(5.8) — otherwise, if `TD` is a specialization of `basic_string_view` or `basic_string` and `TD::value_type` is `char` type, initializes value with `basic_string_view<char type>(v.data(), v.size())`;

(5.9) — otherwise, if `decay_t<TD>` is `char type*` or `const char type*`, initializes value with `static_cast<const char type*>(v)`;

(5.10) — otherwise, if `is_void_v<remove_pointer_t<TD>>` is true or `is_null_pointer_v<TD>` is true, initializes value with `static_cast<const void*>(v)`;

(5.11) — otherwise, initializes value with `handle(v)`.

~~[-?-[Note: Constructing `basic_format_arg` from a pointer to a member is ill-formed unless the user provides an enabled specialization of `formatter` for that pointer to member type. — end note]~~

~~explicit basic_format_arg(float n) noexcept;
explicit basic_format_arg(double n) noexcept;
explicit basic_format_arg(long double n) noexcept;~~

~~-6- Effects: Initializes value with n.~~

~~explicit basic_format_arg(const char type* s) noexcept;~~

~~-7- Preconditions: s points to a NUL-terminated string.~~

~~-8- Effects: Initializes value with s.~~

~~template<class traits>
explicit basic_format_arg(basic_string_view<char type, traits> s) noexcept;~~

~~-9- Effects: Initializes value with basic_string_view<char type>(s.data(), s.size()).~~

~~template<class traits, class Allocator>
explicit basic_format_arg(
const basic_string<char type, traits, Allocator>& s) noexcept;~~

~~-10- Effects: Initializes value with basic_string_view<char type>(s.data(), s.size()).~~

~~explicit basic_format_arg(nullptr_t) noexcept;~~

~~-11- Effects: Initializes value with static_cast<const void*>(nullptr).~~

~~template<class T> explicit basic_format_arg(T* p) noexcept;~~

~~-12- Constraints: is_void_v<T> is true.~~

~~-13- Effects: Initializes value with p.~~

~~-14- [Note: Constructing `basic_format_arg` from a pointer to a member is ill-formed unless the user provides an enabled specialization of `formatter` for that pointer to member type. — end note]~~

3. Modify 22.14.8.1 [format.arg] p17 and p18 as indicated:

template<class T> explicit handle(T&& v) noexcept;

-17- Let

— (17.1) `TD` be `remove_cvref<const> T`,

— (17.2) `const_formattable` be true if `typename Context::template formatter_type<TD>().format(declval<const TD&>(), declval<Context&>())` is well-formed, otherwise false;

— (17.3) `TQ` be `const TD` if `const_formattable` is true, `const TD` satisfies `formattable-with<Context>` and `TD` otherwise.

-18- Mandates: `const_formattable || !is_const_v<remove_reference_t<T>>` is true, `TQ` satisfies `formattable-with<Context>`.

-19- Effects: Initializes `ptr_` with `addressof(val)` and `format_` with


```

    [](basic_format_parse_context<char_type>& parse_ctx,
      Context& format_ctx, const void* ptr) {
      typename Context::template formatter_type<TD> f;
      parse_ctx.advance_to(f.parse(parse_ctx));
      format_ctx.advance_to(f.format(*const_cast<TQ*>(static_cast<const TD*>(ptr)),
                                   format_ctx));
    }

```

4. Modify 22.14.8.2 [\[format.arg.store\]](#) p2 as indicated:

```

template<class Context = format_context, class... Args>
format_arg_store<Context, Args...> make_format_args(Args&&... fmt_args);

```

-2- *Preconditions*: The type `typename Context::template formatter_type<remove\_cvref\_t<Ti>>` meets the *BasicFormatter* requirements (22.14.6.1 [\[formatter.requirements\]](#)) for each T_i in `Args`.

3645. `resize_and_overwrite` is overspecified to call its callback with lvalues

Section: 23.4.3.5 [\[string.capacity\]](#) **Status:** [Immediate](#) **Submitter:** Arthur O'Dwyer **Opened:** 2021-11-28 **Last modified:** 2023-02-09

Priority: 2

View all other [issues](#) in [\[string.capacity\]](#).

Discussion:

23.4.3.5 [\[string.capacity\]](#) p7 says:

- [Let] OP be the expression `std::move(op)(p, n)`.
- [*Precondition*:] OP does not throw an exception or modify p or n .

Notice that p and n above are lvalue expressions.

Discussed with Mark Zeren, Casey Carter, Jonathan Wakely. We observe that:

A. This wording requires vendors to reject

```
s.resize_and_overwrite(100, [](char*&&, size_t&&){ return 0; });
```

which is surprising.

B. This wording requires vendors to accept

```
s.resize_and_overwrite(100, [](char*&, size_t&){ return 0; });
```

which is even more surprising, and also threatens to allow the user to corrupt the internal state (which is why we need to specify the *Precondition* above).

C. A user who writes

```
s.resize_and_overwrite(100, [](auto&&, auto&&){ return 0; });
```

can detect that they're being passed lvalues instead of rvalues. If we change the wording to permit implementations to pass either lvalues or rvalues (their choice), then this will be detectable by the user, so we don't want that if we can help it.

- X. We want to enable implementations to say `move(op)(__p, __n)` and then use `__p` and `__n`.
- Y. We have one implementation which wants to say `move(op)(data(), __n)`, which is not currently allowed, but arguably should be.
- Z. We have to do or say something about disallowing writes to any internal state to which op might get a reference.

Given all of this, Mark and Arthur think that the simplest way out is to say that the arguments are prvalues. It prevents X, but fixes the surprises in A, B, Y, Z. We could do this in the *Let* bullets. Either like so:

- [Let] p be a prvalue of type `charT*` ...
- m be a prvalue of type `size_type` equal to n ,
- OP be the expression `std::move(op)(p, m)`.

or (Arthur's preference) by specifying prvalues in the expression OP itself:

- [Let] OP be the expression `std::move(op)(auto(p), auto(n))`.

No matter which specification approach we adopt, we can also simplify the *Preconditions* bullet to:

- [*Precondition*:] OP does not throw an exception.

because once the user is receiving prvalue copies, it will no longer be physically possible for the user to modify the library's original variables p and n .

[2021-11-29; Arthur O'Dwyer provides wording]

[2022-01-30; Reflector poll]

Set priority to 2 after reflector poll.

Previous resolution [SUPERSEDED]:

This wording is relative to [N4901](#).

1. Modify 23.4.3.5 [\[string.capacity\]](#) as indicated:

```
template<class Operation> constexpr void resize_and_overwrite(size_type n, Operation op);
```

-7- Let

(7.1) — $o = \text{size}()$ before the call to `resize_and_overwrite`.

(7.2) — k be $\min(o, n)$.

(7.3) — p be a `charT*`, such that the range $[p, p + n]$ is valid and `this->compare(0, k, p, k) == 0` is true before the call. The values in the range $[p + k, p + n]$ may be indeterminate (6.7.4 [\[basic.indet\]](#)).

(7.4) — OP be the expression `std::move(op)(auto\(p\), auto\(n\))`.

(7.5) — $r = OP$.

-8- *Mandates*: OP has an integer-like type (25.3.4.4 [\[iterator.concept.winc\]](#)).

-9- *Preconditions*:

(9.1) — OP does not throw an exception ~~or modify p or n~~ .

(9.2) — $r \geq 0$.

(9.3) — $r \leq n$.

(9.4) — After evaluating OP there are no indeterminate values in the range $[p, p + r]$.

-10- *Effects*: Evaluates OP , replaces the contents of `*this` with $[p, p + r]$, and invalidates all pointers and references to the range $[p, p + n]$.

-11- *Recommended practice*: Implementations should avoid unnecessary copies and allocations by, for example, making p a pointer into internal storage and by restoring `*(p + r)` to `charT()` after evaluating OP .

[2023-01-11; Jonathan Wakely provides new wording requested by LWG]

[Issaquah 2023-02-07; LWG]

Move to Immediate for C++23

Proposed resolution:

This wording is relative to [N4917](#).

1. Modify 23.4.3.5 [\[string.capacity\]](#) as indicated:

```
template<class Operation> constexpr void resize_and_overwrite(size_type n, Operation op);
```

-7- Let

(7.1) — $o = \text{size}()$ before the call to `resize_and_overwrite`.

(7.2) — k be $\min(o, n)$.

(7.3) — p be a [value of type](#) `charT*` [or charT* const](#), such that the range $[p, p + n]$ is valid and `this->compare(0, k, p, k) == 0` is true before the call. The values in the range $[p + k, p + n]$ may be indeterminate (6.7.4 [\[basic.indet\]](#)).

[\(7.2\) — m be a value of type size_type or const size_type equal to n.](#)

(7.4) — OP be the expression `std::move(op)(p, m)`.

(7.5) — $r = OP$.

-8- *Mandates*: OP has an integer-like type (25.3.4.4 [\[iterator.concept.winc\]](#)).

-9- *Preconditions*:

(9.1) — OP does not throw an exception or modify p or [m](#).

(9.2) — $r \geq 0$.

(9.3) — $r \leq$ [m](#).

(9.4) — After evaluating OP there are no indeterminate values in the range $[p, p + r]$.

-10- *Effects*: Evaluates *OP*, replaces the contents of **this* with $[p, p + r)$, and invalidates all pointers and references to the range $[p, p + n]$.

-11- *Recommended practice*: Implementations should avoid unnecessary copies and allocations by, for example, making *p* a pointer into internal storage and by restoring $*(p + r)$ to `charT()` after evaluating *OP*.

3655. The *INVOKE* operation and union types

Section: 22.10.4 [func.require] Status: [Immediate](#) Submitter: Jiang An Opened: 2021-12-29 Last modified: 2023-02-07

Priority: 3

View all other [issues](#) in [func.require].

Discussion:

There are two cases of the *INVOKE* operation specified with `std::is_base_of_v` (22.10.4 [func.require] (1.1), (1.4)), which means the following code snippet is ill-formed, as `std::is_base_of_v<B, D>` is false when either *B* or *D* is a union type.

```
union Foo { int x; };
static_assert(std::is_invocable_v<int Foo::*, Foo&>);
```

Currently libstdc++ accepts this code, because it uses slightly different conditions that handle union types. libc++ and MSVC STL reject this code as specified in 22.10.4 [func.require].

Should we change the conditions in 22.10.4 [func.require] (1.1) and (1.4) to match libstdc++ and correctly handle union types?

[2022-01-30; Reflector poll]

Set priority to 3 after reflector poll.

[2023-02-07; Jonathan adds wording]

This is a regression introduced by LWG 2219. In C++14 `std::result_of<int Foo::*(Foo&>::type` was valid, because the *INVOKE* wording used to say "*f* is a pointer to member data of a class *T* and *t1* is an object of type *T* or a reference to an object of type *T* or a reference to an object of a type derived from *T*". Since LWG 2219 we use `is_base_of` which is always false for union types. I don't think LWG 2219 intended to break this case, so we should fix it.

Previous resolution [SUPERSEDED]:

This wording is relative to [N4928](#).

1. Modify 22.10.4 [func.require] as indicated:

-1- Define *INVOKE*(*f*, *t1*, *t2*, ..., *tN*) as follows:

- (1.1) — (*t1*.**f*)(*t2*, ..., *tN*) when *f* is a pointer to a member function of a class *T* and `is_same_v<T, remove_cvref_t<decltype(t1)>>, is_base_of_v<T, remove_reference_t<decltype(t1)>>` is true;
- (1.2) — (*t1*.get().**f*)(*t2*, ..., *tN*) when *f* is a pointer to a member function of a class *T* and `remove_cvref_t<decltype(t1)>` is a specialization of `reference_wrapper`;
- (1.3) — ((**t1*).**f*)(*t2*, ..., *tN*) when *f* is a pointer to a member function of a class *T* and *t1* does not satisfy the previous two items;
- (1.4) — *t1*.**f* when *N* == 1 and *f* is a pointer to data member of a class *T* and `is_same_v<T, remove_cvref_t<decltype(t1)>>, is_base_of_v<T, remove_reference_t<decltype(t1)>>` is true;
- (1.5) — *t1*.get().**f* when *N* == 1 and *f* is a pointer to data member of a class *T* and `remove_cvref_t<decltype(t1)>` is a specialization of `reference_wrapper`;
- (1.6) — (**t1*).**f* when *N* == 1 and *f* is a pointer to data member of a class *T* and *t1* does not satisfy the previous two items;
- (1.7) — *f*(*t1*, *t2*, ..., *tN*) in all other cases.

[2023-02-07; Jonathan provides wording change requested by LWG]

Change `remove_reference_t` to `remove_cvref_t`. `is_base_of` ignores cv-qualifiers, so this isn't necessary, but just using the same transformation in both cases seems simpler to grok.

[Issaquah 2023-02-07; LWG]

Move to Immediate for C++23

Proposed resolution:

This wording is relative to [N4928](#).

1. Modify 22.10.4 [func.require] as indicated:

-1- Define *INVOKE*(*f*, *t1*, *t2*, ..., *tN*) as follows:

- (1.1) — (*t1*.**f*)(*t2*, ..., *tN*) when *f* is a pointer to a member function of a class *T* and `is_same_v<T, remove_cvref_t<decltype(t1)>>, is_base_of_v<T, remove_cvref_t<decltype(t1)>>` is true;

- (1.2) — $(t1.get().*f)(t2, \dots, tN)$ when f is a pointer to a member function of a class T and `remove_cvref_t<decltype(t1)>` is a specialization of `reference_wrapper`;
- (1.3) — $((*t1).*f)(t2, \dots, tN)$ when f is a pointer to a member function of a class T and $t1$ does not satisfy the previous two items;
- (1.4) — $t1.*f$ when $N == 1$ and f is a pointer to data member of a class T and `is_same_v<T, remove_cvref_t<decltype(t1)>> || is_base_of_v<T, remove_cvref_t<decltype(t1)>>` is true;
- (1.5) — $t1.get().*f$ when $N == 1$ and f is a pointer to data member of a class T and `remove_cvref_t<decltype(t1)>` is a specialization of `reference_wrapper`;
- (1.6) — $(*t1).*f$ when $N == 1$ and f is a pointer to data member of a class T and $t1$ does not satisfy the previous two items;
- (1.7) — $f(t1, t2, \dots, tN)$ in all other cases.

3723. `priority_queue::push_range` needs to `append_range`

Section: 24.6.7.4 [[prerequisite.members](#)] **Status:** [Immediate](#) **Submitter:** Casey Carter **Opened:** 2022-06-20 **Last modified:** 2023-02-08

Priority: 2

Discussion:

The `push_range` members of the `queue` (24.6.6.4 [[queue.mod](#)]) and `stack` (24.6.8.5 [[stack.mod](#)]) container adaptors are both specified as "*Effects:* Equivalent to `c.append_range(std::forward<R>(rg))` if that is a valid expression, otherwise `ranges::copy(rg, back_inserter(c))`". For `priority_queue`, however, we have instead (24.6.7.4 [[prerequisite.members](#)]):

-3- *Effects:* Insert all elements of `rg` in `c`.

-4- *Postconditions:* `is_heap(c.begin(), c.end(), comp)` is true.

Since `append_range` isn't one of the operations required of the underlying container, "Insert all elements of `rg`" must be implemented via potentially less efficient means. It would be nice if this `push_back` could take advantage of `append_range` when it's available just as do the other two overloads.

[2022-07-08; Reflector poll]

Set priority to 2 after reflector poll.

[Issaquah 2023-02-08; LWG]

Unanimous consent to move to Immediate.

Proposed resolution:

This wording is relative to [N4910](#).

1. Modify 24.6.7.4 [[prerequisite.members](#)] as indicated:

```
template<container-compatible-range<T> R>
void push_range(R&& rg);
```

-3- *Effects:* Insert all elements of `rg` in `c` via `c.append_range(std::forward<R>(rg))` if that is a valid expression, or `ranges::copy(rg, back_inserter(c))` otherwise. Then restores the heap property as if by `make_heap(c.begin(), c.end(), comp)`.

-4- *Postconditions:* `is_heap(c.begin(), c.end(), comp)` is true.

3734. Inconsistency in `inout_ptr` and `out_ptr` for empty case

Section: 20.3.4.1 [[out_ptr.t](#)] **Status:** [Immediate](#) **Submitter:** Doug Cook **Opened:** 2022-07-11 **Last modified:** 2023-02-09

Priority: 2

Discussion:

`out_ptr` and `inout_ptr` are inconsistent when a pointer-style function returns `nullptr`.

- `out_ptr` leaves the stale value in `smart` (not the value returned by the pointer-style function).
- `inout_ptr` (as resolved by LWG [3594](#)) leaves `nullptr` in `smart` (the value returned by the pointer-style function).

Assume we have the following pointer-style functions that return `nullptr` in case of failure:

```
void ReplaceSomething(/*INOUT*/ int** pp) {
    delete *pp;
    *pp = nullptr;
    return; // Failure!
}

void GetSomething(/*OUT*/ int** pp) {
    *pp = nullptr;
    return; // Failure!
}
```

In the scenario that led to the creation of issue LWG [3594](#):

```
// Before the call, inout contains a stale value.
auto inout = std::make_unique<int>(1);
ReplaceSomething(std::inout_ptr(inout));
// (1) If ReplaceSomething failed (returned nullptr), what does inout contain?
```

Assuming LWG 3594 is resolved as suggested, inout will be empty. (The original N4901 text allows inout to be either empty or to hold a pointer to already-deleted memory.) Using the resolution suggested by LWG 3594, it expands to something like the following (simplified to ignore exceptions and opting to perform the release() before the ReplaceSomething() operation):

```
// Before the call, inout contains a stale value.
auto inout = std::make_unique<int>(1);
int* p = inout.release();
ReplaceSomething(&p);
if (p) {
    inout.reset(p);
}
// (1) If ReplaceSomething failed (returned nullptr), inout contains nullptr.
```

This behavior seems reasonable.

Now consider the corresponding scenario with out_ptr:

```
// Before the call, out contains a stale value.
auto out = std::make_unique<int>(2);
GetSomething(std::out_ptr(out));
// (2) If GetSomething failed (returned nullptr), what does out contain?
```

Based on N4901, out contains the stale value (from make_unique), not the nullptr value returned by GetSomething(). The N4901 model (simplified to ignore exceptions) expands to the following:

```
// Before the call, out contains a stale value.
auto out = std::make_unique<int>(2);
int* p{};
GetSomething(&p);
if (p) {
    out.reset(p);
}
// (2) If GetSomething failed (returned nullptr), out contains a pointer to "2".
```

This behavior seems incorrect to me. It is inconsistent with the behavior of inout_ptr and it is inconsistent with my expectation that out should contain the value returned by GetSomething(), even if that value is nullptr. Intuitively, I expect it to behave as if the out.reset(p) were unconditional.

The reset(p) is conditional as an optimization for cases where reset is non-trivial. For example, shared_ptr's reset(p) requires the allocation of a control block even if p is nullptr. As such, simply making the reset unconditional may be sub-optimal.

I see two primary options for making out_ptr's behavior consistent with inout_ptr:

- Perform an unconditional out.reset() or out = Smart() in the out_ptr_t constructor.
- Add an else clause to the if statement, containing out.reset() or out = Smart().

I note that these solutions do not make use of the additional args..., leaving the out pointer in an empty state. This is analogous to the corresponding state in the similar inout scenario where the inout pointer is left empty as a result of the call to smart.release().

I favor the first resolution, freeing any existing value in the out_ptr_t constructor.

[2022-08-23; Reflector poll]

Set priority to 2 after reflector poll. "A bit like design."

[Issaquah 2023-02-07; LWG]

Move to Immediate for C++23

Proposed resolution:

This wording is relative to N4910.

1. Modify 20.3.4.1 [out_ptr.t] as indicated:

```
explicit out_ptr_t(Smart& smart, Args... args);
```

-6- Effects: Initializes s with smart, a with std::forward<Args>(args) ..., and value-initializes p. Then, equivalent to:

- (6.1) — s.reset();
if the expression s.reset() is well-formed;
- (6.2) — otherwise,
s = Smart();
if is_constructible v<Smart> is true;
- (6.3) — otherwise, the program is ill-formed.

-7- [Note 2: The constructor is not noexcept to allow for a variety of non-terminating and safe implementation strategies. For example, an implementation can allocate a shared_ptr's internal node in the constructor and let implementation-defined exceptions escape safely. The destructor can then move the allocated control block in directly and avoid any other exceptions. — end note]

3772. repeat_view's piecewise constructor is missing *Postconditions*

Section: 26.6.5.2 [[range.repeat.view](#)] Status: [Immediate](#) Submitter: Hewill Kang Opened: 2022-09-12 Last modified: 2023-02-09

Priority: 2

View all other [issues](#) in [[range.repeat.view](#)].

Discussion:

The first two value-bound pair constructors of repeat_view have the *Preconditions* that the integer-like object bound must be non-negative. However, the piecewise constructor does not mention the valid values for bound_args. It would be nice to add a *Postconditions* that the initialized bound_ must be greater than or equal to 0 here.

[2022-09-23; Reflector poll]

Set priority to 2 after reflector poll.

This is trying to state a requirement on users, but that's not what *Postconditions*: means. Should be something more like:

Precondition: If Bound is not unreachable_sentinel_t, the bound_ ≥ 0 after its initialization from bound_args.

Previous resolution [SUPERSEDED]:

This wording is relative to [N4917](#).

1. Modify 26.6.5.2 [[range.repeat.view](#)] as shown:

```
template<class... WArgs, class... BoundArgs>
    requires constructible_from<W, WArgs...> &&
             constructible_from<Bound, BoundArgs...>
constexpr explicit repeat_view(piecewise_construct_t,
    tuple<WArgs...> value_args, tuple<BoundArgs...> bound_args = tuple<>{});
```

-5- *Effects*: Initializes *value_* with arguments of types WArgs... obtained by forwarding the elements of value_args and initializes *bound_* with arguments of types BoundArgs... obtained by forwarding the elements of bound_args. (Here, forwarding an element x of type U within a tuple object means calling std::forward<U>(x).)

-?- *Postconditions*: If Bound is not unreachable_sentinel_t, bound_ ≥ 0.

[2022-09-23; Jonathan provides improved wording]

Previous resolution [SUPERSEDED]:

This wording is relative to [N4917](#).

1. Modify 26.6.5.2 [[range.repeat.view](#)] as shown:

```
template<class... WArgs, class... BoundArgs>
    requires constructible_from<W, WArgs...> &&
             constructible_from<Bound, BoundArgs...>
constexpr explicit repeat_view(piecewise_construct_t,
    tuple<WArgs...> value_args, tuple<BoundArgs...> bound_args = tuple<>{});
```

-?- *Preconditions*: Bound is unreachable_sentinel_t, or the initialization of bound_ yields a non-negative value.

-5- *Effects*: Initializes *value_* with arguments of types WArgs... obtained by forwarding the elements of value_args and initializes *bound_* with arguments of types BoundArgs... obtained by forwarding the elements of bound_args. (Here, forwarding an element x of type U within a tuple object means calling std::forward<U>(x).)

[2023-01-11; Jonathan provides new wording requested by LWG]

[Issaquah 2023-02-07; LWG]

Move to Immediate for C++23

Proposed resolution:

This wording is relative to [N4917](#).

1. Modify 26.6.5.2 [[range.repeat.view](#)] as shown:

```
template<class... TArgs, class... BoundArgs>
    requires constructible_from<T, TArgs...> &&
             constructible_from<Bound, BoundArgs...>
constexpr explicit repeat_view(piecewise_construct_t,
    tuple<TArgs...> value_args, tuple<BoundArgs...> bound_args = tuple<>{});
```

-5- Effects: Initializes `value_` with arguments of types `TArgs...` obtained by forwarding the elements of `value_args` make from `tuple<T>(std::move(value_args))` and initializes `bound_` with arguments of types `BoundArgs...` obtained by forwarding the elements of `bound_args`. (Here, forwarding an element `x` of type `U` within a tuple object means calling `std::forward<U>(x)`.) make from `tuple<Bound>(std::move(bound_args))`. The behavior is undefined if `Bound` is not unreachable sentinel `t` and `bound` is negative.

3786. Flat maps' deduction guide needs to default Allocator to be useful

Section: 24.6.9.2 [flat.map.defn], 24.6.10.2 [flat.multimap.defn] **Status:** Immediate **Submitter:** Johel Ernesto Guerrero Peña **Opened:** 2022-09-25 **Last modified:** 2023-02-09

Priority: 2

Discussion:

This originated from the editorial issue #5800.

P0429R9 added some deduction guides with a non-defaulted Allocator template parameter and a corresponding function parameter that is defaulted. Since the template parameter Allocator is not defaulted, these deduction guides are never used.

[2022-09-28; LWG telecon]

We should not just ignore the allocator, it should be rebound and used for the two container types.

Previous resolution [SUPERSEDED]:

This wording is relative to N4917.

1. Modify 24.6.9.2 [flat.map.defn], class template `flat_map` synopsis, as indicated:

```
[...]
template<ranges::input_range R, class Compare = less<range-key-type<R>>,
        class Allocator = allocator<void>>
    flat_map(from_range_t, R&&, Compare = Compare(), Allocator = Allocator())
    -> flat_map<range-key-type<R>, range-mapped-type<R>, Compare>;
[...]
```

2. Modify 24.6.10.2 [flat.multimap.defn], class template `flat_multimap` synopsis, as indicated:

```
[...]
template<ranges::input_range R, class Compare = less<range-key-type<R>>,
        class Allocator = allocator<void>>
    flat_multimap(from_range_t, R&&, Compare = Compare(), Allocator = Allocator())
    -> flat_multimap<range-key-type<R>, range-mapped-type<R>, Compare>;
[...]
```

[2022-10-19; Jonathan provides improved wording]

Previous resolution [SUPERSEDED]:

This wording is relative to N4917.

1. Modify 24.6.1 [container.adaptors.general] as indicated:

-8- The following exposition-only alias templates may appear in deduction guides for container adaptors:

```
template<class Container>
    using cont-key-type =                    // exposition only
        remove_const_t<typename Container::value_type::first_type>;
template<class Container>
    using cont-mapped-type =                // exposition only
        typename Container::value_type::second_type;
template<class Allocator, class T>
    using alloc-rebind =                    // exposition only
        typename allocator_traits<Allocator>::template rebind_alloc<T>;
```

2. Modify 24.6.9.2 [flat.map.defn], class template `flat_map` synopsis, as indicated:

```
[...]
template<ranges::input_range R, class Compare = less<range-key-type<R>>,
        class Allocator = allocator<void>>
    flat_map(from_range_t, R&&, Compare = Compare(), Allocator = Allocator())
    -> flat_map<range-key-type<R>, range-mapped-type<R>, Compare,
        vector<range-key-type<R>, alloc-rebind<Allocator, range-key-type<R>>>,
        vector<range-mapped-type<R>, alloc-rebind<Allocator, range-mapped-type<R>>>>;

template<ranges::input_range R, class Allocator>
    flat_map(from_range_t, R&&, Allocator)
    -> flat_map<range-key-type<R>, range-mapped-type<R>, less<range-key-type<R>>,
        vector<range-key-type<R>, alloc-rebind<Allocator, range-key-type<R>>>,
        vector<range-mapped-type<R>, alloc-rebind<Allocator, range-mapped-type<R>>>>;
[...]
```

3. Modify 24.6.10.2 [flat_multimap.defn], class template flat_multimap synopsis, as indicated:

```
[...]
template<ranges::input_range R, class Compare = less<range-key-type<R>>,
        class Allocator = allocator<void>>
    flat_multimap(from_range_t, R&&, Compare = Compare(), Allocator = Allocator())
    -> flat_multimap<range-key-type<R>, range-mapped-type<R>, Compare,
        vector<range-key-type<R>, alloc-rebind<Allocator, range-key-type<R>>>,
        vector<range-mapped-type<R>, alloc-rebind<Allocator, range-mapped-type<R>>>>;

template<ranges::input_range R, class Allocator>
    flat_multimap(from_range_t, R&&, Allocator)
    -> flat_multimap<range-key-type<R>, range-mapped-type<R>, less<range-key-type<R>>,
        vector<range-key-type<R>, alloc-rebind<Allocator, range-key-type<R>>>,
        vector<range-mapped-type<R>, alloc-rebind<Allocator, range-mapped-type<R>>>>;

[...]
```

4. Modify 24.6.11.2 [flat_set.defn], class template flat_set synopsis, as indicated:

```
[...]
template<ranges::input_range R, class Compare = less<ranges::range_value_t<R>>,
        class Allocator = allocator<ranges::range_value_t<R>>>
    flat_set(from_range_t, R&&, Compare = Compare(), Allocator = Allocator())
    -> flat_set<ranges::range_value_t<R>, Compare,
        vector<ranges::range_value_t<R>, alloc-rebind<Allocator, ranges::range_value_t<R>>>>;

template<ranges::input_range R, class Allocator>
    flat_set(from_range_t, R&&, Allocator)
    -> flat_set<ranges::range_value_t<R>, less<ranges::range_value_t<R>>,
        vector<ranges::range_value_t<R>, alloc-rebind<Allocator, ranges::range_value_t<R>>>>;

[...]
```

5. Modify 24.6.12.2 [flat_multiset.defn], class template flat_multiset synopsis, as indicated:

```
[...]
template<ranges::input_range R, class Compare = less<ranges::range_value_t<R>>,
        class Allocator = allocator<ranges::range_value_t<R>>>
    flat_multiset(from_range_t, R&&, Compare = Compare(), Allocator = Allocator())
    -> flat_multiset<ranges::range_value_t<R>, Compare,
        vector<ranges::range_value_t<R>, alloc-rebind<Allocator, ranges::range_value_t<R>>>>;

template<ranges::input_range R, class Allocator>
    flat_multiset(from_range_t, R&&, Allocator)
    -> flat_multiset<ranges::range_value_t<R>, less<ranges::range_value_t<R>>,
        vector<ranges::range_value_t<R>, alloc-rebind<Allocator, ranges::range_value_t<R>>>>;

[...]
```

[2023-01-11; Jonathan Wakely provides improved wording]

During LWG telecon Tim pointed out that because allocator<void> does not meet the Cpp17Allocator requirements, it might not "qualify as an allocator" and so would cause the deduction guides to not participate in overload resolution, as per 24.6.1 [container.adaptors.general] p6 (6.4). Use allocator<byte> instead.

[Issaquah 2023-02-07; LWG]

Edited proposed resolution to fix missing `>` in guides for maps. Move to Immediate for C++23

Proposed resolution:

This wording is relative to [N4917](#).

1. Modify 24.6.1 [container.adaptors.general] as indicated:

-8- The following exposition-only alias template may appear in deduction guides for container adaptors:

```
template<class Allocator, class T>
    using alloc-rebind = // exposition only
        typename allocator_traits<Allocator>::template rebind alloc<T>;
```

2. Modify 24.6.9.2 [flat_map.defn], class template flat_map synopsis, as indicated:

```
[...]
template<ranges::input_range R, class Compare = less<range-key-type<R>>,
        class Allocator = allocator<byte>>
    flat_map(from_range_t, R&&, Compare = Compare(), Allocator = Allocator())
    -> flat_map<range-key-type<R>, range-mapped-type<R>, Compare,
        vector<range-key-type<R>, alloc-rebind<Allocator, range-key-type<R>>>,
        vector<range-mapped-type<R>, alloc-rebind<Allocator, range-mapped-type<R>>>>;

template<ranges::input_range R, class Allocator>
    flat_map(from_range_t, R&&, Allocator)
    -> flat_map<range-key-type<R>, range-mapped-type<R>, less<range-key-type<R>>,
        vector<range-key-type<R>, alloc-rebind<Allocator, range-key-type<R>>>,
        vector<range-mapped-type<R>, alloc-rebind<Allocator, range-mapped-type<R>>>>;

[...]
```

3. Modify 24.6.10.2 [flat_multimap.defn], class template flat_multimap synopsis, as indicated:

```
[...]
template<ranges::input_range R, class Compare = less<range-key-type<R>>,
```



```

        class Allocator = allocator<byte>
flat_multimap(from_range_t, R&&, Compare = Compare(), Allocator = Allocator())
-> flat_multimap<range-key-type<R>, range-mapped-type<R>, Compare,
    vector<range-key-type<R>, alloc-rebind<Allocator, range-key-type<R>>>,
    vector<range-mapped-type<R>, alloc-rebind<Allocator, range-mapped-type<R>>>>;

template<ranges::input_range R, class Allocator>
flat_multimap(from_range_t, R&&, Allocator)
-> flat_multimap<range-key-type<R>, range-mapped-type<R>, less<range-key-type<R>>,
    vector<range-key-type<R>, alloc-rebind<Allocator, range-key-type<R>>>,
    vector<range-mapped-type<R>, alloc-rebind<Allocator, range-mapped-type<R>>>>;
[...]
```

4. Modify 24.6.11.2 [\[flat.set.defn\]](#), class template `flat_set` synopsis, as indicated:

```

[...]
```

```

template<ranges::input_range R, class Compare = less<ranges::range_value_t<R>>,
        class Allocator = allocator<ranges::range_value_t<R>>>
flat_set(from_range_t, R&&, Compare = Compare(), Allocator = Allocator())
-> flat_set<ranges::range_value_t<R>, Compare,
    vector<ranges::range_value_t<R>, alloc-rebind<Allocator, ranges::range_value_t<R>>>>;

template<ranges::input_range R, class Allocator>
flat_set(from_range_t, R&&, Allocator)
-> flat_set<ranges::range_value_t<R>, less<ranges::range_value_t<R>>,
    vector<ranges::range_value_t<R>, alloc-rebind<Allocator, ranges::range_value_t<R>>>>;
[...]
```

5. Modify 24.6.12.2 [\[flat.multiset.defn\]](#), class template `flat_multiset` synopsis, as indicated:

```

[...]
```

```

template<ranges::input_range R, class Compare = less<ranges::range_value_t<R>>,
        class Allocator = allocator<ranges::range_value_t<R>>>
flat_multiset(from_range_t, R&&, Compare = Compare(), Allocator = Allocator())
-> flat_multiset<ranges::range_value_t<R>, Compare,
    vector<ranges::range_value_t<R>, alloc-rebind<Allocator, ranges::range_value_t<R>>>>;

template<ranges::input_range R, class Allocator>
flat_multiset(from_range_t, R&&, Allocator)
-> flat_multiset<ranges::range_value_t<R>, less<ranges::range_value_t<R>>,
    vector<ranges::range_value_t<R>, alloc-rebind<Allocator, ranges::range_value_t<R>>>>;
[...]
```

3803. `flat_foo` constructors taking `KeyContainer` lack `KeyCompare` parameter

Section: 24.6.9 [\[flat.map\]](#), 24.6.10 [\[flat.multimap\]](#), 24.6.11 [\[flat.set\]](#), 24.6.12 [\[flat.multiset\]](#) **Status:** [Immediate](#) **Submitter:** Arthur O'Dwyer **Opened:** 2022-10-25
Last modified: 2023-02-10

Priority: 1

View other [active issues](#) in `[flat.map]`.

View all other [issues](#) in `[flat.map]`.

Discussion:

`flat_set`'s current constructor overload set has these two overloads:

```

explicit flat_set(container_type cont);
template<class Allocator>
    flat_set(const container_type& cont, const Allocator& a);
```

I believe it should have these two in addition:

```

flat_set(const key_compare& comp, container_type cont);
template<class Allocator>
    flat_set(const key_compare& comp, const container_type& cont, const Allocator& a);
```

with corresponding deduction guides. Similar wording changes would have to be made to all the `flat_foo` containers.

Tony Table:

```

struct LessWhenDividedBy {
    int divisor_;
    bool operator()(int x, int y) const { return x/divisor_ < y/divisor_; }
};
std::flat_set<int, LessWhenDividedBy> s(data.begin(), data.end(), LessWhenDividedBy(10));
// BEFORE AND AFTER: okay, but cumbersome
std::flat_set<int, LessWhenDividedBy> s(data);
// BEFORE AND AFTER: oops, this default-constructs the comparator

std::flat_set<int, LessWhenDividedBy> s(LessWhenDividedBy(10), data);
// BEFORE: fails to compile
// AFTER: compiles successfully
```

[2022-11-04; Reflector poll]

Set priority to 1 after reflector poll.

[2023-02-09 Tim adds wording]

For each construction that takes containers, this wording allow a comparator to be specified as well. Differing from the suggestion in the issue, the comparator goes last (but before the allocator), for consistency with every other constructor of `flat_meow` taking a comparator. (This is inconsistent with `priority_queue`, but consistency among the type's own constructors seems more important.)

[Issaquah 2023-02-09; LWG]

Move to Immediate for C++23

Proposed resolution:

This wording is relative to [N4928](#).

1. Add a new bullet to 24.6.1 [\[container.adaptors.general\]](#) p6 as indicated:

-6- A deduction guide for a container adaptor shall not participate in overload resolution if any of the following are true:

~~— (6.?) It has both `KeyContainer` and `Compare` template parameters, and is invocable `v<const Compare&, const typename KeyContainer::value_type&, const typename KeyContainer::value_type&>` is not a valid expression or is false.~~

2. Modify 24.6.9.2 [\[flat.map.defn\]](#) as indicated:

```
namespace std {
    template<class Key, class T, class Compare = less<Key>,
            class KeyContainer = vector<Key>, class MappedContainer = vector<T>>
    class flat_map {
    public:
        [...]
        // 24.6.9.3 \[flat.map.cons\], construct/copy/destroy
        flat_map() : flat_map(key_compare()) { }

        flat_map(key_container_type key_cont, mapped_container_type mapped_cont,
            const key_compare& comp = key_compare(););
        template<class Allocator>
        flat_map(const key_container_type& key_cont, const mapped_container_type& mapped_cont,
            const Allocator& a);
        template<class Allocator>
        flat_map(const key_container_type& key_cont, const mapped_container_type& mapped_cont,
        const key_compare& comp, const Allocator& a);

        flat_map(sorted_unique_t, key_container_type key_cont, mapped_container_type mapped_cont,
            const key_compare& comp = key_compare(););
        template<class Allocator>
        flat_map(sorted_unique_t, const key_container_type& key_cont,
            const mapped_container_type& mapped_cont, const Allocator& a);
        template<class Allocator>
        flat_map(sorted_unique_t, const key_container_type& key_cont,
        const mapped_container_type& mapped_cont,
        const key_compare& comp, const Allocator& a);

        [...]
    };

    template<class KeyContainer, class MappedContainer, class Compare = less<typename KeyContainer::value_type>>
    flat_map(KeyContainer, MappedContainer, Compare = Compare();)
    -> flat_map<typename KeyContainer::value_type, typename MappedContainer::value_type,
        Compare=less<typename KeyContainer::value_type>, KeyContainer, MappedContainer>;

    template<class KeyContainer, class MappedContainer, class Allocator>
    flat_map(KeyContainer, MappedContainer, Allocator)
    -> flat_map<typename KeyContainer::value_type, typename MappedContainer::value_type,
        less<typename KeyContainer::value_type>, KeyContainer, MappedContainer>;
    template<class KeyContainer, class MappedContainer, class Compare, class Allocator>
    flat_map(KeyContainer, MappedContainer, Compare, Allocator)
    -> flat_map<typename KeyContainer::value_type, typename MappedContainer::value_type,
        Compare, KeyContainer, MappedContainer>;

    template<class KeyContainer, class MappedContainer, class Compare = less<typename KeyContainer::value_type>>
    flat_map(sorted_unique_t, KeyContainer, MappedContainer, Compare = Compare();)
    -> flat_map<typename KeyContainer::value_type, typename MappedContainer::value_type,
        Compare=less<typename KeyContainer::value_type>, KeyContainer, MappedContainer>;

    template<class KeyContainer, class MappedContainer, class Allocator>
    flat_map(sorted_unique_t, KeyContainer, MappedContainer, Allocator)
    -> flat_map<typename KeyContainer::value_type, typename MappedContainer::value_type,
        less<typename KeyContainer::value_type>, KeyContainer, MappedContainer>;
    template<class KeyContainer, class MappedContainer, class Compare, class Allocator>
    flat_map(sorted_unique_t, KeyContainer, MappedContainer, Compare, Allocator)
    -> flat_map<typename KeyContainer::value_type, typename MappedContainer::value_type,
        Compare, KeyContainer, MappedContainer>;

    [...]
}
```

3. Modify 24.6.9.3 [\[flat.map.cons\]](#) as indicated:

```
flat_map(key_container_type key_cont, mapped_container_type mapped_cont,
    const key_compare& comp = key_compare(););
```

-1- Effects: Initializes `c.keys` with `std::move(key_cont)`, ~~and~~ `c.values` with `std::move(mapped_cont)`, ~~and compare with comp~~, ~~value initializes compare~~; sorts the range `[begin(), end())` with respect to `value_comp()`; and finally erases the duplicate elements

as if by:

```
auto zv = ranges::zip_view(c.keys, c.values);
auto it = ranges::unique(zv, key_equiv(compare)).begin();
auto dist = distance(zv.begin(), it);
c.keys.erase(c.keys.begin() + dist, c.keys.end());
c.values.erase(c.values.begin() + dist, c.values.end());
```

-2- *Complexity*: Linear in N if the container arguments are already sorted with respect to `value_comp()` and otherwise $N \log N$, where N is the value of `key_cont.size()` before this call.

```
template<class Allocator>
flat_map(const key_container_type& key_cont, const mapped_container_type& mapped_cont,
         const Allocator& a);
template<class Allocator>
flat_map(const key_container_type& key_cont, const mapped_container_type& mapped_cont,
         const key_compare& comp, const Allocator& a);
```

-3- *Constraints*: `uses_allocator_v<key_container_type, Allocator>` is true and `uses_allocator_v<mapped_container_type, Allocator>` is true.

-4- *Effects*: Equivalent to `flat_map(key_cont, mapped_cont)` and `flat_map(key_cont, mapped_cont, comp)`, respectively, except that `c.keys` and `c.values` are constructed with `uses-allocator` construction (20.2.8.2 [allocator.uses.construction]).

-5- *Complexity*: Same as `flat_map(key_cont, mapped_cont)` and `flat_map(key_cont, mapped_cont, comp)`, respectively.

```
flat_map(sorted_unique_t, key_container_type key_cont, mapped_container_type mapped_cont,
         const key_compare& comp = key_compare());
```

-6- *Effects*: Initializes `c.keys` with `std::move(key_cont)`, and `c.values` with `std::move(mapped_cont)`, and compare with `comp`; value-initializes `compare`.

-7- *Complexity*: Constant.

```
template<class Allocator>
flat_map(sorted_unique_t s, const key_container_type& key_cont,
         const mapped_container_type& mapped_cont, const Allocator& a);
template<class Allocator>
flat_map(sorted_unique_t s, const key_container_type& key_cont,
         const mapped_container_type& mapped_cont, const key_compare& comp,
         const Allocator& a);
```

-8- *Constraints*: `uses_allocator_v<key_container_type, Allocator>` is true and `uses_allocator_v<mapped_container_type, Allocator>` is true.

-9- *Effects*: Equivalent to `flat_map(s, key_cont, mapped_cont)` and `flat_map(s, key_cont, mapped_cont, comp)`, respectively, except that `c.keys` and `c.values` are constructed with `uses-allocator` construction (20.2.8.2 [allocator.uses.construction]).

-10- *Complexity*: Linear.

4. Modify 24.6.10.2 [flat.multimap.defn] as indicated:

```
namespace std {
template<class Key, class T, class Compare = less<Key>,
         class KeyContainer = vector<Key>, class MappedContainer = vector<T>>
class flat_multimap {
public:
    [...]
    // 24.6.10.3 [flat.multimap.cons], construct/copy/destroy
    flat_multimap() : flat_multimap(key_compare()) { }

    flat_multimap(key_container_type key_cont, mapped_container_type mapped_cont,
                 const key_compare& comp = key_compare());
    template<class Allocator>
    flat_multimap(const key_container_type& key_cont, const mapped_container_type& mapped_cont,
                 const Allocator& a);
    template<class Allocator>
    flat_multimap(const key_container_type& key_cont, const mapped_container_type& mapped_cont,
                 const key_compare& comp, const Allocator& a);

    flat_multimap(sorted_equivalent_t,
                 key_container_type key_cont, mapped_container_type mapped_cont,
                 const key_compare& comp = key_compare());
    template<class Allocator>
    flat_multimap(sorted_equivalent_t, const key_container_type& key_cont,
                 const mapped_container_type& mapped_cont, const Allocator& a);
    template<class Allocator>
    flat_multimap(sorted_equivalent_t, const key_container_type& key_cont,
                 const mapped_container_type& mapped_cont,
                 const key_compare& comp, const Allocator& a);
    [...]
};

template<class KeyContainer, class MappedContainer, class Compare = less<typename KeyContainer::value_type>
flat_multimap(KeyContainer, MappedContainer, Compare = Compare())
-> flat_multimap<typename KeyContainer::value_type, typename MappedContainer::value_type,
Compare=less<typename KeyContainer::value_type>, KeyContainer, MappedContainer>;

template<class KeyContainer, class MappedContainer, class Allocator>
flat_multimap(KeyContainer, MappedContainer, Allocator)
-> flat_multimap<typename KeyContainer::value_type, typename MappedContainer::value_type,
```

```

        less<typename KeyContainer::value_type>, KeyContainer, MappedContainer>;
template<class KeyContainer, class MappedContainer, class Compare, class Allocator>
    flat_multimap(KeyContainer, MappedContainer, Compare, Allocator)
-> flat_multimap<typename KeyContainer::value_type, typename MappedContainer::value_type,
    Compare, KeyContainer, MappedContainer>;

template<class KeyContainer, class MappedContainer, class Compare = less<typename KeyContainer::value_type>
    flat_multimap(sorted_equivalent_t, KeyContainer, MappedContainer, Compare = Compare())
-> flat_multimap<typename KeyContainer::value_type, typename MappedContainer::value_type,
    Compare, less<typename KeyContainer::value_type>, KeyContainer, MappedContainer>;

template<class KeyContainer, class MappedContainer, class Allocator>
    flat_multimap(sorted_equivalent_t, KeyContainer, MappedContainer, Allocator)
-> flat_multimap<typename KeyContainer::value_type, typename MappedContainer::value_type,
    less<typename KeyContainer::value_type>, KeyContainer, MappedContainer>;
template<class KeyContainer, class MappedContainer, class Compare, class Allocator>
    flat_multimap(sorted_equivalent_t, KeyContainer, MappedContainer, Compare, Allocator)
-> flat_multimap<typename KeyContainer::value_type, typename MappedContainer::value_type,
    Compare, KeyContainer, MappedContainer>;
[...]
```

5. Modify 24.6.10.3 [flat_multimap.cons] as indicated:

```

flat_multimap(key_container_type key_cont, mapped_container_type mapped_cont,
    const key_compare& comp = key_compare());
```

-1- *Effects*: Initializes c.keys with std::move(key_cont), and c.values with std::move(mapped_cont), and compare with comp, value-initializes compare; and sorts the range [begin(), end()) with respect to value_comp().

-2- *Complexity*: Linear in N if the container arguments are already sorted with respect to value_comp() and otherwise $N \log N$, where N is the value of key_cont.size() before this call.

```

template<class Allocator>
    flat_multimap(const key_container_type& key_cont, const mapped_container_type& mapped_cont,
        const Allocator& a);
template<class Allocator>
    flat_multimap(const key_container_type& key_cont, const mapped_container_type& mapped_cont,
        const key_compare& comp, const Allocator& a);
```

-3- *Constraints*: uses_allocator_v<key_container_type, Allocator> is true and uses_allocator_v<mapped_container_type, Allocator> is true.

-4- *Effects*: Equivalent to flat_multimap(key_cont, mapped_cont) and flat_multimap(key_cont, mapped_cont, comp), respectively, except that c.keys and c.values are constructed with uses-allocator construction (20.2.8.2 [allocator.uses.construction]).

-5- *Complexity*: Same as flat_multimap(key_cont, mapped_cont) and flat_multimap(key_cont, mapped_cont, comp), respectively.

```

flat_multimap(sorted_equivalent_t, key_container_type key_cont, mapped_container_type mapped_cont,
    const key_compare& comp = key_compare());
```

-6- *Effects*: Initializes c.keys with std::move(key_cont), and c.values with std::move(mapped_cont), and compare with comp, value-initializes compare.

-7- *Complexity*: Constant.

```

template<class Allocator>
    flat_multimap(sorted_equivalent_t s, const key_container_type& key_cont,
        const mapped_container_type& mapped_cont, const Allocator& a);
template<class Allocator>
    flat_multimap(sorted_equivalent_t s, const key_container_type& key_cont,
        const mapped_container_type& mapped_cont, const key_compare& comp,
        const Allocator& a);
```

-8- *Constraints*: uses_allocator_v<key_container_type, Allocator> is true and uses_allocator_v<mapped_container_type, Allocator> is true.

-9- *Effects*: Equivalent to flat_multimap(s, key_cont, mapped_cont) and flat_multimap(s, key_cont, mapped_cont, comp), respectively, except that c.keys and c.values are constructed with uses-allocator construction (20.2.8.2 [allocator.uses.construction]).

-10- *Complexity*: Linear.

6. Modify 24.6.11.2 [flat_set.defn] as indicated:

```

namespace std {
    template<class Key, class Compare = less<Key>, class KeyContainer = vector<Key>>
    class flat_set {
    public:
        [...]

        // [flat.set.cons], constructors
        flat_set() : flat_set(key_compare()) { }

        explicit flat_set(container_type cont, const key_compare& comp = key_compare());
        template<class Allocator>
            flat_set(const container_type& cont, const Allocator& a);
        template<class Allocator>
            flat_set(const container_type& cont, const key_compare& comp, const Allocator& a);
```

```

flat_set(sorted_unique_t, container_type cont, const key_compare& comp = key_compare())
: c(std::move(cont)), compare(comp) { }
template<class Allocator>
flat_set(sorted_unique_t, const container_type& cont, const Allocator& a);
template<class Allocator>
flat_set(sorted_unique_t, const container_type& cont,
const key_compare& comp, const Allocator& a);

[...];
};

template<class KeyContainer, class Compare = less<typename KeyContainer::value_type>>
flat_set(KeyContainer, Compare = Compare())
-> flat_set<typename KeyContainer::value_type, Compare, KeyContainer>;
template<class KeyContainer, class Allocator>
flat_set(KeyContainer, Allocator)
-> flat_set<typename KeyContainer::value_type, less<typename KeyContainer::value_type>, KeyContainer>;
template<class KeyContainer, class Compare, class Allocator>
flat_set(KeyContainer, Compare, Allocator)
-> flat_set<typename KeyContainer::value_type, Compare, KeyContainer>;

template<class KeyContainer, class Compare = less<typename KeyContainer::value_type>>
flat_set(sorted_unique_t, KeyContainer, Compare = Compare())
-> flat_set<typename KeyContainer::value_type, Compare, KeyContainer>;
template<class KeyContainer, class Allocator>
flat_set(sorted_unique_t, KeyContainer, Allocator)
-> flat_set<typename KeyContainer::value_type, less<typename KeyContainer::value_type>, KeyContainer>;
template<class KeyContainer, class Compare, class Allocator>
flat_set(sorted_unique_t, KeyContainer, Compare, Allocator)
-> flat_set<typename KeyContainer::value_type, Compare, KeyContainer>;

[...];
}

```

7. Modify 24.6.11.3 [flat.set.cons] as indicated:

```

flat_set(container_type cont, const key_compare& comp = key_compare());

```

-1- *Effects*: Initializes `c` with `std::move(cont)` and `compare` with `comp`, value-initializes `compare`, sorts the range `[begin(), end())` with respect to `compare`, and finally erases all but the first element from each group of consecutive equivalent elements.

-2- *Complexity*: Linear in N if `cont` is already sorted with respect to `compare` and otherwise $N \log N$, where N is the value of `cont.size()` before this call.

```

template<class Allocator>
flat_set(const container_type& cont, const Allocator& a);
template<class Allocator>
flat_set(const container_type& cont, const key_compare& comp, const Allocator& a);

```

-3- *Constraints*: `uses_allocator_v<container_type, Allocator>` is true.

-4- *Effects*: Equivalent to `flat_set(cont)` and `flat_set(cont, comp)`, respectively, except that `c` is constructed with uses-allocator construction (20.2.8.2 [allocator.uses.construction]).

-5- *Complexity*: Same as `flat_set(cont)` and `flat_set(cont, comp)`, respectively.

```

template<class Allocator>
flat_set(sorted_unique_t s, const container_type& cont, const Allocator& a);
template<class Allocator>
flat_set(sorted_unique_t s, const container_type& cont, const key_compare& comp, const Allocator& a);

```

-6- *Constraints*: `uses_allocator_v<container_type, Allocator>` is true.

-7- *Effects*: Equivalent to `flat_set(s, cont)` and `flat_set(s, cont, comp)`, respectively, except that `c` is constructed with uses-allocator construction (20.2.8.2 [allocator.uses.construction]).

-8- *Complexity*: Linear.

8. Modify 24.6.12.2 [flat.multiset.defn] as indicated:

```

namespace std {
template<class Key, class Compare = less<Key>, class KeyContainer = vector<Key>>
class flat_multiset {
public:
[...];

// [flat.multiset.cons], constructors
flat_multiset() : flat_multiset(key_compare()) { }

explicit flat_multiset(container_type cont, const key_compare& comp = key_compare());
template<class Allocator>
flat_multiset(const container_type& cont, const Allocator& a);
template<class Allocator>
flat_multiset(const container_type& cont, const key_compare& comp, const Allocator& a);

flat_multiset(sorted_equivalent_t, container_type cont, const key_compare& comp = key_compare())
: c(std::move(cont)), compare(comp) { }
template<class Allocator>
flat_multiset(sorted_equivalent_t, const container_type& cont, const Allocator& a);
template<class Allocator>

```

```

flat multiset(sorted equivalent t, const container_type& cont,
              const key_compare& comp, const Allocator& a);

[...]
};

template<class KeyContainer, class Compare = less<typename KeyContainer::value_type>>
flat multiset(KeyContainer, Compare = Compare())
-> flat multiset<typename KeyContainer::value_type, Compare, KeyContainer>;
template<class KeyContainer, class Allocator>
flat multiset(KeyContainer, Allocator)
-> flat multiset<typename KeyContainer::value_type, less<typename KeyContainer::value_type>, KeyContainer>;
template<class KeyContainer, class Compare, class Allocator>
flat multiset(KeyContainer, Compare, Allocator)
-> flat multiset<typename KeyContainer::value_type, Compare, KeyContainer>;

template<class KeyContainer, class Compare = less<typename KeyContainer::value_type>>
flat multiset(sorted equivalent t, KeyContainer, Compare = Compare())
-> flat multiset<typename KeyContainer::value_type, Compare, KeyContainer>;
template<class KeyContainer, class Allocator>
flat multiset(sorted equivalent t, KeyContainer, Allocator)
-> flat multiset<typename KeyContainer::value_type, less<typename KeyContainer::value_type>, KeyContainer>;
template<class KeyContainer, class Compare, class Allocator>
flat multiset(sorted equivalent t, KeyContainer, Compare, Allocator)
-> flat multiset<typename KeyContainer::value_type, Compare, KeyContainer>;

[...]
}

```

9. Modify 24.6.12.3 [\[flat_multiset.cons\]](#) as indicated:

```
flat_multiset(container_type cont, const key_compare& comp = key_compare());
```

-1- *Effects*: Initializes *c* with `std::move(cont)` and *compare* with *comp*, and sorts the range `[begin(), end())` with respect to *compare*.

-2- *Complexity*: Linear in *N* if *cont* is already sorted with respect to *compare* and otherwise *N* log *N*, where *N* is the value of `cont.size()` before this call.

```

template<class Allocator>
flat_multiset(const container_type& cont, const Allocator& a);
template<class Allocator>
flat multiset(const container_type& cont, const key_compare& comp, const Allocator& a);

```

-3- *Constraints*: `uses_allocator_v<container_type, Allocator>` is true.

-4- *Effects*: Equivalent to `flat_multiset(cont)` and `flat_multiset(cont, comp)`, respectively, except that *c* is constructed with `uses_allocator` construction (20.2.8.2 [\[allocator.uses.construction\]](#)).

-5- *Complexity*: Same as `flat_multiset(cont)` and `flat_multiset(cont, comp)`, respectively.

```

template<class Allocator>
flat_multiset(sorted equivalent_t s, const container_type& cont, const Allocator& a);
template<class Allocator>
flat multiset(sorted equivalent_t s, const container_type& cont, const key_compare& comp, const Allocator& a);

```

-6- *Constraints*: `uses_allocator_v<container_type, Allocator>` is true.

-7- *Effects*: Equivalent to `flat_multiset(s, cont)` and `flat_multiset(s, cont, comp)`, respectively, except that *c* is constructed with `uses_allocator` construction (20.2.8.2 [\[allocator.uses.construction\]](#)).

-8- *Complexity*: Linear.

3810. CTAD for `std::basic_format_args`

Section: 22.14.8.3 [\[format.args\]](#) **Status:** [Immediate](#) **Submitter:** Jonathan Wakely **Opened:** 2022-11-03 **Last modified:** 2023-02-07

Priority: 3

View all other [issues](#) in `[format.args]`.

Discussion:

It seems desirable for this should work:

```

auto args_store = std::make_format_args<C>(1,2,3);
// ...
std::basic_format_args args = args_store;

```

i.e. CTAD should deduce the Context argument from the `fmt-store-args<C, int, int, int>` object returned by `make_format_args`.

Another example (from Tomasz Kamiński):

Given:

```

template<typename Context>
void foo(basic_format_args<Context> c);

```

```
foo(make_format_args<SomeContext>(...)); // won't work
foo(basic_format_args(make_format_args<SomeContext>(...))); // should work
```

Since `fmt-store-args` is exposition-only, it's not entirely clear that it must have exactly the form shown in 22.14.8.2 [\[format.arg.store\]](#). E.g. maybe it can have different template arguments, or could be a nested type defined inside `basic_format_args`. I don't know how much of the exposition-only spec is actually required for conformance. If CTAD is already intended to be required, it's a bit subtle.

If we want the CTAD to work (and I think it's nice if it does) we could make that explicit by adding a deduction guide.

[Kona 2022-11-12; Set priority to 3, status to LEWG]

[2023-01-10; LEWG telecon]

Unanimous consensus in favor.

[Issaquah 2023-02-06; LWG]

Unanimous consent (9/0/0) to move to Immediate for C++23.

Proposed resolution:

This wording is relative to [N4917](#).

1. Modify 22.14.8.3 [\[format.args\]](#) as indicated:

```
namespace std {
    template<class Context>
    class basic_format_args {
        size_t size_; // exposition only
        const basic_format_arg<Context>* data_; // exposition only

    public:
        basic_format_args() noexcept;

        template<class... Args>
        basic_format_args(const format_arg_store<Context, Args...& store) noexcept;

        basic_format_arg<Context> get(size_t i) const noexcept;
    };

    template<class Context, class... Args>
    basic_format_args(format_arg_store<Context, Args...>) -> basic_format_args<Context>;
}
```

3827. Deprecate `<stdalign.h>` and `<stdbool.h>` macros

Section: 17.14.4 [\[stdalign.h.syn\]](#), 17.14.5 [\[stdbool.h.syn\]](#) **Status:** [Immediate](#) **Submitter:** GB **Opened:** 2022-11-10 **Last modified:** 2023-02-10

Priority: Not Prioritized

Discussion:

This is the resolution for NB comments:

- [GB-081](#) 17.14.4 [\[stdalign.h.syn\]](#) Deprecate contents of `<stdalign.h>`.
- [GB-082](#) 17.14.5 [\[stdbool.h.syn\]](#) Deprecate contents of `<stdbool.h>`.

GB-081:

C2x defines `alignas` as a keyword, so `<stdalign.h>` is empty in C2x. C++23 should deprecate the `__alignas_is_defined` macro now, rather than wait until a future C++ standard is based on C2x. That gives users longer to prepare for the removal of the macro.

Recommended change: Deprecate `__alignas_is_defined` and move it to Annex D. Maybe keep a note in 17.14.4 [\[stdalign.h.syn\]](#) that the macro is present but deprecated.

GB-082:

C2x supports `bool` as a built-in type, and `true` and `false` as keywords. Consequently, C2x marks the `__bool_true_false_are_defined` as obsolescent. C++23 should deprecate that attribute now, rather than wait until a future C++ standard is based on C2x. That gives users longer to prepare for the removal of the macro.

Recommended change: Deprecate `__bool_true_false_are_defined` and move it to Annex D. Maybe keep a note in 17.14.5 [\[stdbool.h.syn\]](#) that the macro is present but deprecated.

[Kona 2022-11-10; Jonathan provides wording]

[Kona 2022-11-10; Waiting for LEWG electronic polling]

[2022-11-08; Kona LEWG]

Strong consensus to accept GB-81. Strong consensus to accept GB-82.

Previous resolution [SUPERSEDED]:

This wording is relative to [N4917](#).

1. Modify 17.14.4 [\[stdalign.h.syn\]](#) as indicated:

17.14.4 Header <stdalign.h> synopsis [stdalign.h.syn]

~~#define __alignas_is_defined 1~~

-1- The contents of the C++ header <stdalign.h> are the same as the C standard library header <stdalign.h>, with the following changes: The header <stdalign.h> does not define a macro named alignas. The macro alignas is defined ([depr.c.macros]) is deprecated.

SEE ALSO: ISO C 7.15

2. Modify 17.14.5 [\[stdbool.h.syn\]](#) as indicated:

17.14.5 Header <stdbool.h> synopsis [stdbool.h.syn]

~~#define __bool_true_false_are_defined 1~~

-1- The contents of the C++ header <stdbool.h> are the same as the C standard library header <stdbool.h>, with the following changes: The header <stdbool.h> does not define macros named bool, true, or false. The macro bool true false are defined ([depr.c.macros]) is deprecated.

SEE ALSO: ISO C 7.18

3. Add a new subclause to Annex D [\[depr\]](#) between D.10 [\[depr.res.on.required\]](#) and D.11 [\[depr.relops\]](#), with this content:

D?? Deprecated C macros [depr.c.macros]

-1- The header <stdalign.h> has the following macro:

#define alignas is defined 1

-2- The header <stdbool.h> has the following macro:

#define bool true false are defined 1

[Issaquah 2023-02-06; LWG]

Green text additions in Clause 17 was supposed to be adding notes. Drop them instead. Unanimous consent (14/0/0) to move to Immediate for C++23.

Proposed resolution:

This wording is relative to [N4928](#).

1. Modify 17.14.4 [\[stdalign.h.syn\]](#) as indicated:

17.14.4 Header <stdalign.h> synopsis [stdalign.h.syn]

~~#define __alignas_is_defined 1~~

-1- The contents of the C++ header <stdalign.h> are the same as the C standard library header <stdalign.h>, with the following changes: The header <stdalign.h> does not define a macro named alignas.

SEE ALSO: ISO C 7.15

2. Modify 17.14.5 [\[stdbool.h.syn\]](#) as indicated:

17.14.5 Header <stdbool.h> synopsis [stdbool.h.syn]

~~#define __bool_true_false_are_defined 1~~

-1- The contents of the C++ header <stdbool.h> are the same as the C standard library header <stdbool.h>, with the following changes: The header <stdbool.h> does not define macros named bool, true, or false.

SEE ALSO: ISO C 7.18

3. Add a new subclause to Annex D [\[depr\]](#) between D.10 [\[depr.res.on.required\]](#) and D.11 [\[depr.relops\]](#), with this content:

D?? Deprecated C macros [depr.c.macros]

-1- The header <stdalign.h> has the following macro:

#define alignas is defined 1

-2- The header <stdbool.h> has the following macro:

#define bool true false are defined 1

Section: 17.4.1 [[cstdint.syn](#)] **Status:** [Immediate](#) **Submitter:** GB **Opened:** 2022-11-10 **Last modified:** 2023-02-07

Priority: Not Prioritized

View other [active issues](#) in [[cstdint.syn](#)].

View all other [issues](#) in [[cstdint.syn](#)].

Discussion:

This is the resolution for NB comment [GB-080](#) 17.4.1 [[cstdint.syn](#)] Sync `intmax_t` and `uintmax_t` with C2x.

With the approval of [WG14 N2888](#) the next C standard will resolve the long-standing issue that implementations cannot support 128-bit integer types properly without ABI breaks. C++ should adopt the same fix now, rather than waiting until a future C++ standard is rebased on C2x.

31.13.2 [[cinttypes.syn](#)] also mentions those types, but doesn't need a change. The proposed change allows `intmax_t` to be an extended integer type of the same width as `long long`, in which case we'd still want those abs overloads.

Recommended change: Add to 31.13.2 [[cinttypes.syn](#)] p2 "except that `intmax_t` is not required to be able to represent all values of extended integer types wider than `long long`, and `uintmax_t` is not required to be able to represent all values of extended integer types wider than unsigned `long long`."

[Kona 2022-11-10; Waiting for LEWG electronic polling]

[2022-11; LEWG electronic polling]

Unanimous consensus in favor.

[Issaquah 2023-02-06; LWG]

Unanimous consent (13/0/0) to move to Immediate for C++23.

Proposed resolution:

This wording is relative to [N4917](#).

1. Modify 31.13.2 [[cinttypes.syn](#)] as indicated:

-1- The contents and meaning of the header `<cinttypes>` are the same as the C standard library header `<inttypes.h>`, with the following changes:

(1.1) — The header `<cinttypes>` includes the header `<cstdint>` instead of `<stdint.h>`, and

(1.2) — `intmax_t` and `uintmax_t` are not required to be able to represent all values of extended integer types wider than `long long` and unsigned `long long` respectively, and

(1.2) — if and only if the type `intmax_t` designates an extended integer type, the following function signatures are added:

```
intmax_t abs(intmax_t);
imaxdiv_t div(intmax_t, intmax_t);
```

which shall have the same semantics as the function signatures `intmax_t imaxabs(intmax_t)` and `imaxdiv_t imaxdiv(intmax_t, intmax_t)`, respectively.

SEE ALSO: ISO C 7.8

[3833](#). Remove specialization `template<size_t N> struct formatter<const charT[N], charT>`

Section: 22.14.6.3 [[format.formatter.spec](#)] **Status:** [Immediate](#) **Submitter:** Mark de Wever **Opened:** 2022-11-27 **Last modified:** 2023-02-08

Priority: 2

View other [active issues](#) in [[format.formatter.spec](#)].

View all other [issues](#) in [[format.formatter.spec](#)].

Discussion:

In the past I discussed with Victor and Charlie to remove

```
template<size_t N> struct formatter<const charT[N], charT>;
```

Charlie disliked that since MSVC STL already shipped it and Victor mentioned it was useful. Instead of proposing to remove the specialization in LWG [3701](#) ("Make `formatter<remove_cvref_t<const charT[N]>, charT>` requirement explicit") I proposed to keep it. It's unused but it doesn't hurt.

That was until [P2585R0](#) "Improve default container formatting". This paper makes it no longer possible to instantiate this specialization. See [here for an example](#) and a possible work-around.

The relevant wording is 22.14.1 [[format.syn](#)]:

```
template<class R>
constexpr unspecified format_kind = unspecified;

template<ranges::input_range R>
```

```
requires same_as<R, remove_cvref_t<R>>
constexpr range_format format_kind<R> = see below;
```

combined with 22.14.7.1 [\[format.range.fmtkind\]](#) p1:

A program that instantiates the primary template of `format_kind` is ill-formed.

The issue is that `constexpr charT[N]` does not satisfy the requirement `same_as<R, remove_cvref_t<R>>`. So it tries to instantiate the primary template, which is ill-formed.

I see two possible solutions:

1. *Removing* the specialization

```
template<size_t N> struct formatter<constexpr charT[N], charT>;
```

2. *Adding* a specialization

```
template<class charT, size_t N>
constexpr range_format format_kind<constexpr charT[N]> =
    range_format::disabled;
```

I discussed this issue privately and got no objection for solution 1, therefore I propose to take that route. Implementations can still implement solution 2 as an extension until they are ready to ship an API/ABI break.

[2022-11-30; Reflector poll]

Set priority to 2 after reflector poll.

"Rationale is not really convincing why the first option is the right one."

"The point is not that we would need to add a `format_kind` specialization, it is that the specialization is inconsistent with the design of `formatter`, which is supposed to be instantiated only for cv-unqualified non-reference types."

Previous resolution [SUPERSEDED]:

This wording is relative to [N4917](#).

1. Modify 22.14.6.3 [\[format.formatter.spec\]](#) as indicated:

-2- Let `charT` be either `char` or `wchar_t`. Each specialization of `formatter` is either enabled or disabled, as described below. A *debug-enabled* specialization of `formatter` additionally provides a public, `constexpr`, non-static member function `set_debug_format()` which modifies the state of the `formatter` to be as if the type of the *std-format-spec* parsed by the last call to parse were `?`. Each header that declares the template `formatter` provides the following enabled specializations:

(2.1) — The debug-enabled specializations [...]

(2.2) — For each `charT`, the debug-enabled string type specializations

```
template<> struct formatter<char*, charT>;
template<> struct formatter<constexpr char*, charT>;
template<size_t N> struct formatter<charT[N], charT>;
template<size_t N> struct formatter<constexpr charT[N], charT>;
template<class traits, class Allocator>
    struct formatter<basic_string<charT, traits, Allocator>, charT>;
template<class traits>
    struct formatter<basic_string_view<charT, traits>, charT>;
```

(2.3) — [...]

(2.4) — [...]

2. Add a new paragraph to C.1.9 [\[diff.cpp20.utilities\]](#) as indicated:

Affected subclause: 22.14.6.3 [\[format.formatter.spec\]](#)

Change: Remove the `formatter` specialization `template<size_t N> struct formatter<constexpr charT[N], charT>`.

Rationale: The `formatter` specialization was not used in the Standard library. Keeping the specialization well-formed required an additional `format_kind` specialization.

Effect on original feature: Valid C++ 2020 code that instantiated the removed specialization is now ill-formed.

[2023-02-07 Tim provides updated wording]

[Issaquah 2023-02-08; LWG]

Unanimous consent to move to Immediate.

Proposed resolution:

This wording is relative to [N4928](#).

1. Modify 22.14.6.3 [\[format.formatter.spec\]](#) as indicated:

-2- Let `charT` be either `char` or `wchar_t`. Each specialization of `formatter` is either enabled or disabled, as described below. A *debug-enabled* specialization of `formatter` additionally provides a public, `constexpr`, non-static member function `set_debug_format()` which modifies the state of the `formatter` to be as if the type of the *std-format-spec* parsed by the last call to `parse` were `?`. Each header that declares the template `formatter` provides the following enabled specializations:

(2.1) — The debug-enabled specializations [...]

(2.2) — For each `charT`, the debug-enabled string type specializations

```
template<> struct formatter<charT*, charT>;
template<> struct formatter<const charT*, charT>;
template<size_t N> struct formatter<charT[N], charT>;
template<size_t N> struct formatter<const charT[N], charT>;
template<class traits, class Allocator>
    struct formatter<basic_string<charT, traits, Allocator>, charT>;
template<class traits>
    struct formatter<basic_string_view<charT, traits>, charT>;
```

(2.3) — [...]

(2.4) — [...]

2. Add a new paragraph to C.1.9 [\[diff.cpp20.utilities\]](#) as indicated:

Affected subclause: 22.14.6.3 [\[format.formatter.spec\]](#)

Change: Removed the `formatter` specialization `template<size_t N> struct formatter<const charT[N], charT>`.

Rationale: The specialization is inconsistent with the design of `formatter`, which is intended to be instantiated only with cv-unqualified object types.

Effect on original feature: Valid C++ 2020 code that instantiated the removed specialization can become ill-formed.

3836. `std::expected<bool, E1>` conversion constructor `expected(const expected<U, G>&)` should take precedence over `expected(U&&)` with operator `bool`

Section: 22.8.6.2 [\[expected.object.cons\]](#) **Status:** [Immediate](#) **Submitter:** Hui Xie **Opened:** 2022-11-30 **Last modified:** 2023-02-10

Priority: 1

Discussion:

The issue came up when implementing `std::expected` in `libc++`. Given the following example:

```
struct BaseError{};
struct DerivedError : BaseError{};

std::expected<int, DerivedError> e1(5);
std::expected<int, BaseError> e2(e1); // e2 holds 5
```

In the above example, `e2` is constructed with the conversion constructor

```
expected::expected(const expected<U, G>&)
```

and the value 5 is correctly copied into `e2` as `expected`.

However, if we change the type from `int` to `bool`, the behaviour is very surprising.

```
std::expected<bool, DerivedError> e1(false);
std::expected<bool, BaseError> e2(e1); // e2 holds true
```

In this example `e2` is constructed with

```
expected::expected(U&&)
```

together with

```
expected::operator bool() const
```

Instead of copying `e1`'s "false" into `e2`, it uses `operator bool`, which returns `true` in this case and `e2` would hold "true" instead.

This is surprising behaviour given how inconsistent between `int` and `bool`.

The reason why the second example uses a different overload is that the constructor `expected(const expected<U, G>& rhs);` has the following constraint (22.8.6.2 [\[expected.object.cons\]](#) p17):

(17.3) — `is_constructible_v<T, expected<U, G>&>` is false; and

(17.4) — `is_constructible_v<T, expected<U, G>>` is false; and

(17.5) — `is_constructible_v<T, const expected<U, G>&>` is false; and

(17.6) — `is_constructible_v<T, const expected<U, G>>` is false; and

- (17.7) — `is_convertible_v<expected<U, G>&, T>` is false; and
- (17.8) — `is_convertible_v<expected<U, G>&&, T>` is false; and
- (17.9) — `is_convertible_v<const expected<U, G>&, T>` is false; and
- (17.10) — `is_convertible_v<const expected<U, G>&&, T>` is false; and

Since `T` is `bool` in the second example, and `bool` can be constructed from `std::expected`, this overload will be removed. and the overload that takes `U&&` will be selected.

I would suggest to special case `bool`, i.e.

- (The above 8 constraints); or
- `is_same_v<remove_cv_t<T>, bool>` is true

And we need to make sure this overload and the overload that takes `expected(U&&)` be mutually exclusive.

[2023-01-06; Reflector poll]

Set priority to 1 after reflector poll.

There was a mix of votes for P1 and P2 but also one for NAD ("The design of forward/repack construction for `expected` matches optional, when if the stored value can be directly constructed, we use that."). `std::optional<bool>` is similarly affected. Any change should consider the effects on `expected<expected<>>` use cases.

[Issaquah 2023-02-08; Jonathan provides wording]

[Issaquah 2023-02-09; LWG]

Move to Immediate for C++23

Proposed resolution:

This wording is relative to [N4928](#).

1. Modify 22.5.3.2 [optional.ctor] as indicated:

```
template<class U = T> constexpr explicit(see below) optional(U&& v);
```

-23- Constraints:

[Drafting note: Change this paragraph to a bulleted list.]

- (23.1) — `is_constructible_v<T, U>` is true,
- (23.2) — `is_same_v<remove_cvref_t<U>, in_place_t>` is false, and
- (23.3) — `is_same_v<remove_cvref_t<U>, optional>` is false, and
- (23.4) — if `T` is `cv bool`, `remove_cvref_t<U>` is not a specialization of `optional`.

-24- Effects: Direct-non-list-initializes the contained value with `std::forward<U>(v)`.

-25- Postconditions: `*this` has a value.

-26- Throws: Any exception thrown by the selection constructor of `T`.

-27- Remarks: If `T`'s selected constructor is a constexpr constructor, this constructor is a constexpr constructor. The expression inside `explicit` is equivalent to:

```
!is_convertible_v<U, T>
```

```
template<class U> constexpr explicit(see below) optional(const optional<U>& rhs);
```

-28- Constraints:

- (28.1) — `is_constructible_v<T, const U>` is true, and
- (28.1) — if `T` is not `cv bool`, `converts-from-any-cvref<T, optional<U>>` is false.

-29- Effects: If `rhs` contains a value, direct-non-list-initializes the contained value with `*rhs`.

-30- Postconditions: `rhs.has_value() == this->has_value()`.

-31- Throws: Any exception thrown by the selection constructor of `T`.

-32- Remarks: The expression inside `explicit` is equivalent to:

```
!is_convertible_v<const U&, T>
```

```
template<class U> constexpr explicit(see below) optional(optional<U>&& rhs);
```

-33- Constraints:

- (33.1) — `is_constructible_v<T, U>` is true, and

- (33.1) — `if T is not cv bool, converts-from-any-cvref<T, optional<U>>` is false.

-34- *Effects*: If rhs contains a value, direct-non-list-initializes the contained value with `std::move(*rhs)`. `rhs.has_value()` is unchanged.

-35- *Postconditions*: `rhs.has_value() == this->has_value()`.

-36- *Throws*: Any exception thrown by the selection constructor of T.

-37- *Remarks*: The expression inside `explicit` is equivalent to:

```
!is_convertible_v<U, T>
```

2. Modify 22.8.6.2 [\[expected.object.cons\]](#) as indicated:

```
template<class U, class G>
constexpr explicit(see below) expected(const expected<U, G>& rhs);
template<class U, class G>
constexpr explicit(see below) expected(expected<U, G>&& rhs);
```

-17- *Let*:

- (17.1) — UF be `const U&` for the first overload and U for the second overload.
- (17.2) — GF be `const G&` for the first overload and G for the second overload.

-18- *Constraints*:

- (18.1) — `is_constructible_v<T, UF>` is true; and
- (18.2) — `is_constructible_v<E, GF>` is true; and
- (18.3) — `if T is not cv bool, converts-from-any-cvref<T, expected<U, G>>` is false; and
- (18.4) — `is_constructible_v<unexpected<E>, expected<U, G>&>` is false; and
- (18.5) — `is_constructible_v<unexpected<E>, expected<U, G>>` is false; and
- (18.6) — `is_constructible_v<unexpected<E>, const expected<U, G>&>` is false; and
- (18.7) — `is_constructible_v<unexpected<E>, const expected<U, G>>` is false.

-19- *Effects*: If `rhs.has_value()`, direct-non-list-initializes `val` with `std::forward<UF>(*rhs)`. Otherwise, direct-non-list-initializes `unex` with `std::forward<GF>(rhs.error())`.

-20- *Postconditions*: `rhs.has_value()` is unchanged; `rhs.has_value() == this->has_value()` is true.

-21- *Throws*: Any exception thrown by the initialization of `val` or `unex`.

-22- *Remarks*: The expression inside `explicit` is equivalent to `!is_convertible_v<UF, T> || !is_convertible_v<GF, E>`.

```
template<class U = T>
constexpr explicit(!is_convertible_v<U, T>) expected(U&& v);
```

-23- *Constraints*:

- (23.1) — `is_same_v<remove_cvref_t<U>, in_place_t>` is false; and
- (23.2) — `is_same_v<expected, remove_cvref_t<U>>` is false; and
- (23.3) — `is_constructible_v<T, U>` is true; and
- (23.4) — `remove_cvref_t<U>` is not a specialization of `unexpected`; **and**
- (23.5) — `if T is cv bool, remove_cvref_t<U> is not a specialization of expected`.

-24- *Effects*: Direct-non-list-initializes `val` with `std::forward<U>(v)`.

-25- *Postconditions*: `has_value()` is true.

-26- *Throws*: Any exception thrown by the initialization of `val`.

3843. `std::expected<T,E>::value()` & assumes E is copy constructible

Section: 22.8.6.6 [\[expected.object.obs\]](#) **Status:** [Immediate](#) **Submitter:** Jonathan Wakely **Opened:** 2022-12-20 **Last modified:** 2023-02-10

Priority: Not Prioritized

Discussion:

22.8.6.6 [\[expected.object.obs\]](#) p9 says:

Throws: `bad_expected_access(error())` if `has_value()` is false.

But if `error()` returns a reference to a move-only type then it can't be copied and the function body is ill-formed. Should it be constrained with `is_copy_constructible_v<E>`? Or just mandate it?

Similarly, the `value()&&` and `value() const&&` overloads require `is_move_constructible_v<E>` to be true for `bad_expected_access(std::move(error()))` to be valid. Casey Carter pointed out they also require it to be copyable so that the exception can be thrown, as per 14.2 [\[except.throw\]](#) p5.

[Issaquah 2023-02-09; LWG]

Move to Immediate for C++23

Proposed resolution:

1. Modify 22.8.6.6 [\[expected.object.obs\]](#) as indicated:

```
constexpr const T& value() const &;
constexpr T& value() &;
```

-?- Mandates: `is_copy_constructible v<E> is true.`

-8- *Returns:* `val`, if `has_value()` is true.

-9- *Throws:* `bad_expected_access(as_const(error()))` if `has_value()` is false.

```
constexpr T&& value() &&;
constexpr const T&& value() const &&;
```

-?- Mandates: `is_copy_constructible v<E> is true and is_constructible v<E, decltype(std::move(error()))> is true.`

-10- *Returns:* `std::move(val)`, if `has_value()` is true.

-11- *Throws:* `bad_expected_access(std::move(error()))` if `has_value()` is false.

3847. `ranges::to` can still return views

Section: 26.5.7.2 [\[range.utility.conv.to\]](#), 26.5.7.3 [\[range.utility.conv.adaptors\]](#) **Status:** [Immediate](#) **Submitter:** Hewill Kang **Opened:** 2023-01-06 **Last modified:** 2023-02-08

Priority: 2

View other [active issues](#) in [\[range.utility.conv.to\]](#).

View all other [issues](#) in [\[range.utility.conv.to\]](#).

Discussion:

The intention of `ranges::to` is to construct a non-view object, which is reflected in its constraint that object type `C` should not model a view.

The specification allows `C` to be a const-qualified type which does not satisfy view such as `const string_view`, making `ranges::to` return a dangling view. We should ban such cases.

[2023-02-01; Reflector poll]

Set priority to 2 after reflector poll. Just require `C` to be a cv-unqualified object type.

Previous resolution [SUPERSEDED]:

This wording is relative to [N4917](#).

1. Modify 26.2 [\[ranges.syn\]](#) as indicated:

```
#include <compare>           // see 17.11.1 [compare.syn]
#include <initializer_list>   // see 17.10.2 [initializer.list.syn]
#include <iterator>           // see 25.2 [iterator.synopsis]

namespace std::ranges {
    [...]
    // 26.5.7 [range.utility.conv], range conversions
    template<class C, input_range R, class... Args> requires (!view<remove_cv_t<C>>)
        constexpr C to(R&& r, Args&&... args); // freestanding
    template<template<class...> class C, input_range R, class... Args>
        constexpr auto to(R&& r, Args&&... args); // freestanding
    template<class C, class... Args> requires (!view<remove_cv_t<C>>)
        constexpr auto to(Args&&... args); // freestanding
    template<template<class...> class C, class... Args>
        constexpr auto to(Args&&... args); // freestanding
    [...]
}
```

2. Modify 26.5.7.2 [\[range.utility.conv.to\]](#) as indicated:

```
template<class C, input_range R, class... Args> requires (!view<remove_cv_t<C>>)
constexpr C to(R&& r, Args&&... args);
```

-1- *Returns:* An object of type `C` constructed from the elements of `r` in the following manner:

[...]

3. Modify 26.5.7.3 [\[range.utility.conv.adaptors\]](#) as indicated:

```
template<class C, class... Args> requires (!view<remove_cv_t<C>>)
constexpr auto to(Args&&... args);
template<template<class...> class C, class... Args>
constexpr auto to(Args&&... args);
```

-1- Returns: A range adaptor closure object (26.7.2 [\[range.adaptor.object\]](#)) $\mathfrak{f}$ that is a perfect forwarding call wrapper (22.10.4 [\[func.require\]](#)) with the following properties:

[...]

[2023-02-02; Jonathan provides improved wording]

[Issaquah 2023-02-08; LWG]

Unanimous consent to move to Immediate. This also resolves LWG [3787](#).

Proposed resolution:

This wording is relative to [N4928](#).

1. Modify 26.5.7.2 [\[range.utility.conv.to\]](#) as indicated:

```
template<class C, input_range R, class... Args> requires (!view<C>)
constexpr C to(R&& r, Args&&... args);
```

-?- Mandates: C is a cv-unqualified class type.

-1- Returns: An object of type C constructed from the elements of r in the following manner:

[...]

2. Modify 26.5.7.3 [\[range.utility.conv.adaptors\]](#) as indicated:

```
template<class C, class... Args> requires (!view<C>)
constexpr auto to(Args&&... args);
template<template<class...> class C, class... Args>
constexpr auto to(Args&&... args);
```

-?- Mandates: For the first overload, C is a cv-unqualified class type.

-1- Returns: A range adaptor closure object (26.7.2 [\[range.adaptor.object\]](#)) $\mathfrak{f}$ that is a perfect forwarding call wrapper (22.10.4 [\[func.require\]](#)) with the following properties:

[...]

[3862](#). `basic_const_iterator`'s `common_type` specialization is underconstrained

Section: 25.2 [\[iterator.synopsis\]](#) Status: [Immediate](#) Submitter: Hewill Kang Opened: 2023-01-25 Last modified: 2023-02-07

Priority: Not Prioritized

View all other [issues](#) in [\[iterator.synopsis\]](#).

Discussion:

To make `basic_const_iterator` compatible with its unwrapped iterators, the standard defines the following `common_type` specialization:

```
template<class T, common_with<T> U>
struct common_type<basic_const_iterator<T>, U> {
    using type = basic_const_iterator<common_type_t<T, U>>;
};
```

For type U, when it shares a common type with the unwrapped type T of `basic_const_iterator`, the common type of both is `basic_const_iterator` of the common type of T and U.

However, since this specialization only constrains U and T to have a common type, this allows U to be any type that satisfies such requirement, such as `optional<T>`, in which case computing the common type of both would produce a hard error inside the specialization, because `basic_const_iterator` requires the template parameter to be `input_iterator`, while `optional` clearly isn't.

Previous resolution [SUPERSEDED]:

This wording is relative to [N4928](#).

1. Modify 25.2 [\[iterator.synopsis\]](#), header `<iterator>` synopsis, as indicated:

```
#include <compare> // see 17.11.1 \[compare.syn\]
#include <concepts> // see 18.3 \[concepts.syn\]

namespace std {
    [...]
    template<class T, common_with<T> U>
        requires input_iterator<U>
    struct common_type<basic_const_iterator<T>, U> { // freestanding
        using type = basic_const_iterator<common_type_t<T, U>>;
    };
    template<class T, common_with<T> U>
        requires input_iterator<U>
    struct common_type<U, basic_const_iterator<T>> { // freestanding
```

```

        using type = basic_const_iterator<common_type_t<T, U>>;
    };
    [...]
}

```

[2023-02-06; Jonathan provides improved wording based on Casey's suggestion during the prioritization poll.]

[Issaquah 2023-02-07; LWG]

Move to Immediate for C++23

Proposed resolution:

This wording is relative to [N4928](#).

1. Modify 25.2 [\[iterator.synopsis\]](#), header <iterator> synopsis, as indicated:

```

#include <compare>           // see 17.11.1 \[compare.syn\].
#include <concepts>          // see 18.3 \[concepts.syn\].

namespace std {
    [...]
    template<class T, common_with<T> U>
        requires input_iterator<common_type_t<T, U>>
    struct common_type<basic_const_iterator<T>, U> {                // freestanding
        using type = basic_const_iterator<common_type_t<T, U>>;
    };
    template<class T, common_with<T> U>
        requires input_iterator<common_type_t<T, U>>
    struct common_type<U, basic_const_iterator<T>> {                // freestanding
        using type = basic_const_iterator<common_type_t<T, U>>;
    };
    template<class T, common_with<T> U>
        requires input_iterator<common_type_t<T, U>>
    struct common_type<basic_const_iterator<T>, basic_const_iterator<U>> { // freestanding
        using type = basic_const_iterator<common_type_t<T, U>>;
    };
    [...]
}

```

3865. Sorting a range of pairs

Section: 22.3.3 [\[pairs.spec\]](#) **Status:** [Immediate](#) **Submitter:** Barry Revzin **Opened:** 2023-01-28 **Last modified:** 2023-02-09

Priority: 2

View other [active issues](#) in [\[pairs.spec\]](#).

View all other [issues](#) in [\[pairs.spec\]](#).

Discussion:

Consider this example:

```

#include <algorithm>
#include <ranges>

int main() {
    int a[3] = {1, 2, -1};
    int b[3] = {1, 4, 1};
    std::ranges::sort(std::views::zip(a, b));
}

```

This is currently valid C++23 code, but wasn't before [P2165](#) (Compatibility between tuple, pair and *tuple-like* objects). Before P2165, `zip(a, b)` returned a range whose reference was `std::pair<int&, int&>` and whose `value_type` was `std::pair<int, int>` and `std::pair`, unlike `std::tuple`, does not have any heterogeneous comparisons — which is required to satisfy the sortable concept.

While the `zip` family of range adapters no longer has this problem, nothing prevents users from themselves creating a range whose reference type is `pair<T&, U&>` and whose `value_type` is `pair<T, U>` (which is now a valid range after the `zip` paper) and then discovering that this range isn't sortable, even though the equivalent using `tuple` is.

Suggested resolution:

Change `pair`'s comparison operators from comparing two arguments of type `const pair<T1, T2>&` to instead comparing arguments of types `const pair<T1, T2>&` and `const pair<U1, U2>&`.

[2023-02-05; Barry provides wording]

[2023-02-06; Reflector poll]

Set priority to 2 after reflector poll.

[Issaquah 2023-02-07; LWG]

Move to Immediate for C++23

Proposed resolution:

This wording is relative to [N4928](#).

1. Modify 22.2.1 [\[utility.syn\]](#), header <utility> synopsis, as indicated:

```
[...]
// 22.3.3 \[pairs.spec\], pair specialized algorithms
template<class T1, class T2, class U1, class U2>
    constexpr bool operator==(const pair<T1, T2>&, const pair<TU1, TU2>&);
template<class T1, class T2, class U1, class U2>
    constexpr common_comparison_category_t<synth-three-way-result<T1, UU1>,
                                           synth-three-way-result<T2, UU2>>
    operator<=>(const pair<T1, T2>&, const pair<TU1, TU2>&);
[...]
```

2. Modify 22.3.3 [\[pairs.spec\]](#) as indicated:

```
template<class T1, class T2, class U1, class U2>
    constexpr bool operator==(const pair<T1, T2>& x, const pair<TU1, TU2>& y);

-1- [...]

-2- [...]

template<class T1, class T2, class U1, class U2>
    constexpr common_comparison_category_t<synth-three-way-result<T1, UU1>,
                                           synth-three-way-result<T2, UU2>>
    operator<=>(const pair<T1, T2>& x, const pair<TU1, TU2>& y);

-3- [...]
```

3869. Deprecate std::errc constants related to UNIX STREAMS

Section: 19.5.2 [\[system.error.syn\]](#) Status: [Immediate](#) Submitter: Jonathan Wakely Opened: 2023-01-30 Last modified: 2023-02-07

Priority: Not Prioritized

View all other [issues](#) in [\[system.error.syn\]](#).

Discussion:

This is the resolution for NB comment [GB-084](#)

The error numbers ENODATA, ENOSR, ENOSTR and ETIME are all marked "obsoletent" in POSIX 2017 (the current normative reference for C++) and they are absent in the current POSIX 202x draft. They related to the obsolete STREAMS API, which was optional and not required for conformance to the previous POSIX standard (because popular unix-like systems refused to implement it). C++11 added those error numbers to <errno.h> and also defined corresponding errc enumerators: `errc::no_message_available`, `errc::no_stream_resources`, `errc::not_a_stream` and `errc::stream_timeout`.

Given the obsolescent status of those constants in the current normative reference and their absence from the next POSIX standard, WG21 should consider deprecating them now. A deprecation period will allow removing them when C++ is eventually rebased to a new POSIX standard. Otherwise C++ will be left with dangling references to ENODATA, ENOSR, ENOSTR and ETIME that are not defined in the POSIX reference.

After a period of deprecation they can be removed from Annex D, and the names added to 16.4.5.3.2 [\[zombie.names\]](#) so that implementations can continue to define them if they need to.

[Issaquah 2023-02-06; LWG]

Unanimous consent (9/0/0) to move to Immediate for C++23.

Proposed resolution:

This wording is relative to [N4928](#).

1. Modify 19.4.2 [\[cermo.syn\]](#), header <cerrno> synopsis, as indicated:

```
#define ENETUNREACH see below
#define ENFILE see below
#define ENOBUFFS see below
#define ENODATA see below
#define ENODEV see below
#define ENOENT see below
#define ENOEXEC see below
#define ENOLCK see below
#define ENOLINK see below
#define ENOMEM see below
#define ENOMSG see below
#define ENOPROTOPT see below
#define ENOSPC see below
#define ENOSR see below
#define ENOSTR see below
#define ENOSYS see below
#define ENOTCONN see below
#define ENOTDIR see below
#define ENOTEMPTY see below
...
```

```

#define EROFS see below
#define EPIPE see below
#define ESRCH see below
#define ETIME see below
#define ETIMEDOUT see below
#define ETXTBSY see below
#define EWOULDBLOCK see below
#define EXDEV see below

```

-1- The meaning of the macros in this header is defined by the POSIX standard.

2. Modify 19.5.2 [system.error.syn], header <system_error> synopsis, as indicated:

```

no_child_process,          // ECHILD
no_link,                   // ENOLINK
no_lock_available,         // ENOLCK
no_message_available,      // ENODATA
no_message,                // ENOMSG
no_protocol_option,        // ENOPROTOPT
no_space_on_device,        // ENOSPC
no_stream_resources,       // ENOSR
no_such_device_or_address, // ENXIO
no_such_device,            // ENODEV
no_such_file_or_directory, // ENOENT
no_such_process,           // ESRCH
not_a_directory,           // ENOTDIR
not_a_socket,              // ENOTSOCK
not_a_stream,              // ENOSTR
not_connected,             // ENOTCONN
not_enough_memory,         // ENOMEM
...
result_out_of_range,       // ERANGE
state_not_recoverable,     // ENOTRECOVERABLE
stream_timeout,            // ETIME
text_file_busy,            // ETXTBSY
timed_out,                 // ETIMEDOUT

```

3. Modify D [depr], Annex D, Compatibility Features, by adding a new subclause before D.13 [depr.default.allocator]:

D.?? Deprecated error numbers [depr.cerrno]

-1- The following macros are defined in addition to those specified in 19.4.2 [errno.syn]:

```

#define ENODATA see below
#define ENOSR see below
#define ENOSTR see below
#define ETIME see below

```

-2- The meaning of these macros is defined by the POSIX standard.

-4- The following enum `errc` enumerators are defined in addition to those specified in 19.5.2 [system.error.syn]:

```

no_message_available, // ENODATA
no_stream_resources,  // ENOSR
not_a_stream,         // ENOSTR
stream_timeout,       // ETIME

```

-4- The value of each enum `errc` enumerator above is the same as the value of the `<errno>` macro shown in the above synopsis.

3870. Remove `voidify`

Section: 27.11.1 [specialized.algorithms.general] Status: Immediate Submitter: Jonathan Wakely Opened: 2023-01-30 Last modified: 2023-02-10

Priority: Not Prioritized

Discussion:

This is the resolution for NB comment [GB-121](#)

The `voidify` helper breaks const-correctness, for no tangible benefit. C++20 ballot comment US 215 also suggested removing it, but failed to achieve consensus. That should be reconsidered.

The only claimed benefits are:

- Allowing the `uninitialized_xxx` algorithms to create objects in const storage (including overwriting objects declared as const which is usually UB). The caller should be responsible for using `const_cast` if that's really desirable. Implicitly removing 'const' is unsafe and unnecessary.
- Supporting creation of const objects with `std::construct_at`. This seems reasonable, but should be supported by adding a dedicated function that doesn't conflate the type of the storage to write to and the object to create, e.g. `construct_at<const T>(ptr)`.

[Issaquah 2023-02-06; LWG]

Casey noted:

The claimed benefit is allowing the `uninitialized_xxx` algorithms to create objects of `const` and/or `volatile` type, which they cannot otherwise do since they deduce the type of object to be created from the reference type of the pertinent iterator. Creating `const` objects has some (marginal?) benefits over using `const` pointers to mutable objects. For example, their non-mutable members cannot be modified via casting away `const` without undefined behavior. A unit test might take advantage of this behavior to force a compiler to diagnose such undefined behavior in a constant expression.

The issue submitter was aware of this, but an open Core issue, CWG 2514, would invalidate that benefit. If accepted, objects with dynamic storage duration (such as those created by `std::construct_as` and the `std::uninitialized_xxx` algorithms) would never be `const` objects, so casting away the `const` would not be undefined. So implicitly removing `const` in `voidify` would still allow modifying "truly `const`" objects (resulting in undefined behaviour), without being able to create "truly `const`" objects in locations where that actually is safe. If CWG 2514 is accepted, the `voidify` behaviour would be all downside.

LWG requested removing the remaining casts from the proposed resolution, relying on an implicit conversion to `void*` instead. Move to Immediate for C++23.

Previous resolution [SUPERSEDED]:

This wording is relative to [N4928](#).

1. Modify 27.11.1 [\[specialized.algorithms.general\]](#), **General**, as indicated:

-4- Some algorithms specified in 27.11 [\[specialized.algorithms\]](#) make use of the exposition-only function `voidify`:

```
template<class T>
constexpr void* voidify(T& obj) noexcept {
    return const_cast<void*>(static_cast<const volatile void*>(addressof(obj)));
}
```

Proposed resolution:

This wording is relative to [N4928](#).

1. Modify 27.11.1 [\[specialized.algorithms.general\]](#), **General**, as indicated:

-4- Some algorithms specified in 27.11 [\[specialized.algorithms\]](#) make use of the exposition-only function `voidify`:

```
template<class T>
constexpr void* voidify(T& obj) noexcept {
    return const_cast<void*>(static_cast<const volatile void*>(addressof(obj)));
}
```

[3871](#). Adjust note about `terminate`

Section: 16.4.2.5 [\[compliance\]](#) **Status:** [Immediate](#) **Submitter:** CA **Opened:** 2023-02-01 **Last modified:** 2023-02-07

Priority: Not Prioritized

View all other [issues](#) in [\[compliance\]](#).

Discussion:

This is the resolution for NB comment [CA-076](#)

16.4.2.5 [\[compliance\]](#) p4 has this note:

[Note 1: Throwing a standard library provided exception is not observably different from `terminate()` if the implementation does not unwind the stack during exception handling (14.4 [\[except.handle\]](#)) and the user's program contains no catch blocks. — end note]

Even under the conditions described by the note, a call to `terminate()` is observably different from throwing an exception if the current `terminate_handler` function observes what would have been the currently handled exception in the case where the exception was thrown.

The set of conditions should be extended to include something along the lines of "and the current `terminate_handler` function simply calls `abort()`".

Previous resolution [SUPERSEDED]:

This wording is relative to [N4928](#).

1. Modify 16.4.2.5 [\[compliance\]](#), "Freestanding implementations", as indicated:

-4- [Note 1: Throwing a standard library provided exception is not observably different from `terminate()` if the implementation does not unwind the stack during exception handling (14.4 [\[except.handle\]](#)) and the user's program contains no catch blocks and the current `terminate_handler` function simply calls `abort()`. — end note]

[Issaquah 2023-02-06; LWG]

If the note isn't true then remove it.

Poll: keep note and change as proposed? 3/1/10.

Poll: drop the note entirely? 10/0/5.

Drop the note and move to Immediate for C++20: 9/0/2.

Proposed resolution:

This wording is relative to [N4928](#).

1. Modify 16.4.2.5 [\[compliance\]](#), "Freestanding implementations", as indicated:

~~4 [Note 1: Throwing a standard library provided exception is not observably different from `terminate()` if the implementation does not unwind the stack during exception handling (14.4 [\[except.handle\]](#)) and the user's program contains no catch blocks. — end note]~~

3872. `basic_const_iterator` should have custom `iter_move`

Section: 25.5.3 [\[const.iterators\]](#) Status: [Immediate](#) Submitter: Hewill Kang Opened: 2023-01-31 Last modified: 2023-02-10

Priority: 3

View other [active issues](#) in `[const.iterators]`.

View all other [issues](#) in `[const.iterators]`.

Discussion:

The standard does not currently customize `iter_move` for `basic_const_iterator`, which means that applying `iter_move` to `basic_const_iterator` will invoke the default behavior. Although the intent of such an operation is unpredictable, it does introduce some inconsistencies:

```
int x[] = {1, 2, 3};
using R1 = decltype(x | views::as_rvalue | views::as_const);
using R2 = decltype(x | views::as_const | views::as_rvalue);
using Z1 = decltype(views::zip(x) | views::as_rvalue | views::as_const);
using Z2 = decltype(views::zip(x) | views::as_const | views::as_rvalue);

static_assert(same_as<ranges::range_reference_t<R1>, const int&&>);
static_assert(same_as<ranges::range_reference_t<R2>, const int&&>);
static_assert(same_as<ranges::range_reference_t<Z1>, tuple<const int&&>>);
static_assert(same_as<ranges::range_reference_t<Z2>, tuple<const int&&>>); // failed
```

In the above example, `views::zip(x) | views::as_const` will produce a range whose iterator type is `basic_const_iterator` with reference of `tuple<const int&>`. Since `iter_move` adopts the default behavior, its rvalue reference will also be `tuple<const int&>`, so applying `views::as_rvalue` to it won't have any effect.

Such an inconsistency seems undesirable.

The proposed resolution adds an `iter_move` specialization for `basic_const_iterator` and specifies the return type as `common_reference_t<const iter_value_t<It>&&, iter_rvalue_reference_t<It>>`, which is the type that `input_iterator` is guaranteed to be valid. This is also in sync with the behavior of `range-v3`.

[Issaquah 2023-02-10; LWG issue processing]

Move to Immediate for C++23

Proposed resolution:

This wording is relative to [N4928](#).

1. Modify 25.5.3 [\[const.iterators\]](#) as indicated:

```
namespace std {
    template<class I>
        concept not-a-const-iterator = see below;

    template<indirectly_readable I>
        using iter_const_rvalue_reference_t = // exposition only
            common_reference_t<const iter_value_t<I>&&, iter_rvalue_reference_t<I>>>;

    template<input_iterator Iterator>
    class basic_const_iterator {
        Iterator current_ = Iterator(); // exposition only
        using reference = iter_const_reference_t<Iterator>; // exposition only
        using rvalue_reference = iter_const_rvalue_reference_t<Iterator>; // exposition only

    public:
        [...]
        template<sized_sentinel_for<Iterator> S>
            requires different_from<S, basic_const_iterator>
            friend constexpr difference_type operator-(const S& x, const basic_const_iterator& y);
            friend constexpr rvalue_reference iter_move(const basic_const_iterator& i)
                noexcept(noexcept(static_cast<rvalue_reference>(ranges::iter_move(i.current_))));
        [...]
        return static_cast<rvalue_reference>(ranges::iter_move(i.current_));
    };
};
```

3875. `std::ranges::repeat_view<T, IntegerClass>::iterator` may be ill-formed

Section: 26.6.5 [\[range.repeat\]](#) Status: [Immediate](#) Submitter: Jiang An Opened: 2023-02-05 Last modified: 2023-02-10

Priority: Not Prioritized

Discussion:

26.6.5.3 [\[range.repeat.iterator\]](#) specifies `difference_type` as

```
using difference_type = conditional_t<is-signed-integer-like<index-type>,
    index-type,
    IOTA-DIFF-T(index-type)>;
```

which always instantiates `IOTA-DIFF-T(index-type)`, and thus possibly makes the program ill-formed when `index-type` is an integer-class type (`index-type` is same as `Bound` in this case), because `IOTA-DIFF-T(index-type)` is specified to be `iter_difference_t<index-type>` which may be ill-formed (26.6.4.2 [\[range.iota.view\]](#)/1.1).

I think the intent is using `index-type` as-is without instantiating `IOTA-DIFF-T` when `is-signed-integer-like<index-type>` is true.

However, when `Bound` is an unsigned integer-class type, it's unclear which type should the difference type be, or whether `repeat_view` should be well-formed when the possibly intended `IOTA-DIFF-T(Bound)` is ill-formed.

[2023-02-09 Tim adds wording]

We should reject types for which there is no usable difference type.

[Issaquah 2023-02-09; LWG]

Move to Immediate for C++23

Proposed resolution:

This wording is relative to [N4928](#).

1. Modify 26.2 [\[ranges.syn\]](#), header `<ranges>` synopsis, as indicated:

```
// [...]
namespace std::ranges {
    template<move_constructible W, semiregular Bound = unreachable_sentinel_t>
    requires see below(is_object_v<W> && same_as<W, remove_cv_t<W>> &&
    (is_integer_like<Bound> || same_as<Bound, unreachable_sentinel_t>))
    class repeat_view;
```

2. Modify 26.6.5.2 [\[range.repeat.view\]](#) as indicated:

```
namespace std::ranges {

    template<class T>
    concept integer-like-with-usable-difference-type = //exposition only
    is-signed-integer-like<T> || (is-integer-like<T> && weakly_incrementable<T>);

    template<move_constructible W, semiregular Bound = unreachable_sentinel_t>
    requires (is_object_v<W> && same_as<W, remove_cv_t<W>> &&
    (is_integer_likeinteger-like-with-usable-difference-type<Bound> || same_as<Bound, unreachable_sentinel_t>))
    class repeat_view {
    [...]
    };
}
```

3. Modify 26.6.5.3 [\[range.repeat.iterator\]](#) as indicated:

```
namespace std::ranges {
    template<move_constructible W, semiregular Bound = unreachable_sentinel_t>
    requires (is_object_v<W> && same_as<W, remove_cv_t<W>> &&
    (is_integer_likeinteger-like-with-usable-difference-type<Bound> || same_as<Bound, unreachable_sentinel_t>))
    class repeat_view<W, Bound>::iterator {
    private:
        using index-type = // exposition only
        conditional_t<same_as<Bound, unreachable_sentinel_t>, ptrdiff_t, Bound>;
    [...]
    public:
    [...]
        using difference_type = see belowconditional_t<is signed integer like<index-type>,
        index-type,
        IOTA-DIFF-T(index-type)>;
    [...]
    };
}
```

~~?- If `is-signed-integer-like<index-type>` is true, the member typedef-name difference type denotes `index-type`. Otherwise, it denotes `IOTA-DIFF-T(index-type)`. (26.6.4.2 [\[range.iota.view\]](#)).~~

[3876](#). Default constructor of `std::layout_xx::mapping` misses precondition

Section: 24.7.3.4 [\[mdspan.layout\]](#) **Status:** [Immediate](#) **Submitter:** Christian Trott **Opened:** 2023-02-09 **Last modified:** 2023-02-10

Priority: Not Prioritized

Discussion:

As shortly discussed during the LWG review of a `layout_stride` defect, there is currently no protection against creating layout mappings with all static extents where the product of those extents exceeds the representable value range of the `index_type`.

For example, the following statement does not violate any preconditions or mandates:

```
layout_left::mapping<extents<int, 100000, 100000>> a{};
```

But `a.required_span_size()` would overflow since the implied span size is 10B and thus exceeds what `int` can represent.

This is only a problem for all static extents, since with any dynamic extent in the mix the implied span size is 0. Hence we can check for this via a mandates check on the class.

The paper [P2798R0](#) has been provided with the proposed wording as shown below.

[Issaquah 2023-02-10; LWG issue processing]

Move to Immediate for C++23

Proposed resolution:

This wording is relative to [N4928](#).

1. Modify 24.7.3.4.5.1 [\[mdspan.layout.left.overview\]](#) as indicated:

-2- If `Extents` is not a specialization of `extents`, then the program is ill-formed.

-3- `layout_left::mapping<E>` is a trivially copyable type that models regular for each `E`.

-?- Mandates: If `Extents::rank_dynamic() == 0` is true, then the size of the multidimensional index space `Extents()` is representable as a value of type `typename Extents::index_type`.

2. Modify 24.7.3.4.6.1 [\[mdspan.layout.right.overview\]](#) as indicated:

-2- If `Extents` is not a specialization of `extents`, then the program is ill-formed.

-3- `layout_right::mapping<E>` is a trivially copyable type that models regular for each `E`.

-?- Mandates: If `Extents::rank_dynamic() == 0` is true, then the size of the multidimensional index space `Extents()` is representable as a value of type `typename Extents::index_type`.

3. Modify 24.7.3.4.7.1 [\[mdspan.layout.stride.overview\]](#) as indicated:

-2- If `Extents` is not a specialization of `extents`, then the program is ill-formed.

-3- `layout_stride::mapping<E>` is a trivially copyable type that models regular for each `E`.

-?- Mandates: If `Extents::rank_dynamic() == 0` is true, then the size of the multidimensional index space `Extents()` is representable as a value of type `typename Extents::index_type`.

[3877](#). Incorrect constraints on const-qualified monadic overloads for `std::expected`

Section: 22.8.6.7 [\[expected.object.monadic\]](#), 22.8.7.7 [\[expected.void.monadic\]](#) **Status:** [Immediate](#) **Submitter:** Sy Brand **Opened:** 2023-02-09 **Last modified:** 2023-02-10

Priority: Not Prioritized

View other [active issues](#) in [\[expected.object.monadic\]](#).

View all other [issues](#) in [\[expected.object.monadic\]](#).

Discussion:

The constraints for `and_then`, `transform`, `transform_error`, and `or_else` for `std::expected` seem incorrect for const overloads. E.g., from 22.8.6.7 [\[expected.object.monadic\]](#)

```
template<class F> constexpr auto transform(F&& f) &&;
template<class F> constexpr auto transform(F&& f) const &&;
[...]
```

Constraints: `is_move_constructible_v<E>` is true.

That constraint should likely be `is_move_constructible_v<const E>` for the const-qualified version. Same for the lvalue overloads, and for the three other functions, including in the void partial specialization. For example, currently this code would result in a hard compiler error inside the body of `transform` rather than failing the constraint:

```
const std::expected<int, std::unique_ptr<int>> e;
std::move(e).transform([](auto) { return 42; });
```

Previous resolution [SUPERSEDED]:

This wording is relative to [N4928](#).

1. Modify 22.8.6.7 [\[expected.object.monadic\]](#) as indicated:

```
template<class F> constexpr auto and_then(F&& f) &;
template<class F> constexpr auto and_then(F&& f) const &;

[...]

-2- Constraints: is_copy_constructible_v<Edecltype((error().))> is true.

[...]

template<class F> constexpr auto and_then(F&& f) &&;
template<class F> constexpr auto and_then(F&& f) const &&;

[...]

-6- Constraints: is_move_constructible_v<Edecltype((error().))> is true.

[...]

template<class F> constexpr auto or_else(F&& f) &;
template<class F> constexpr auto or_else(F&& f) const &;

[...]

-10- Constraints: is_copy_constructible_v<Fdecltype((value().))> is true.

[...]

template<class F> constexpr auto or_else(F&& f) &&;
template<class F> constexpr auto or_else(F&& f) const &&;

[...]

-14- Constraints: is_move_constructible_v<Fdecltype((value().))> is true.

[...]

template<class F> constexpr auto transform(F&& f) &;
template<class F> constexpr auto transform(F&& f) const &;

[...]

-18- Constraints: is_copy_constructible_v<Edecltype((error().))> is true.

[...]

template<class F> constexpr auto transform(F&& f) &&;
template<class F> constexpr auto transform(F&& f) const &&;

[...]

-22- Constraints: is_move_constructible_v<Edecltype((error().))> is true.

[...]

template<class F> constexpr auto transform_error(F&& f) &;
template<class F> constexpr auto transform_error(F&& f) const &;

[...]

-26- Constraints: is_copy_constructible_v<Fdecltype((value().))> is true.

[...]

template<class F> constexpr auto transform_error(F&& f) &&;
template<class F> constexpr auto transform_error(F&& f) const &&;

[...]

-30- Constraints: is_move_constructible_v<Fdecltype((value().))> is true.
```

2. Modify 22.8.7.7 [\[expected.void.monadic\]](#) as indicated:

```
template<class F> constexpr auto and_then(F&& f) &;
template<class F> constexpr auto and_then(F&& f) const &;

[...]

-2- Constraints: is_copy_constructible_v<Edecltype((error().))> is true.

[...]

template<class F> constexpr auto and_then(F&& f) &&;
template<class F> constexpr auto and_then(F&& f) const &&;

[...]
```

-6- Constraints: is_move_constructible_v<E, decltype((error()))> is true.

[...]

```
template<class F> constexpr auto transform(F&& f) &;  
template<class F> constexpr auto transform(F&& f) const &;
```

[...]

-16- Constraints: is_copy_constructible_v<E, decltype((error()))> is true.

[...]

```
template<class F> constexpr auto transform(F&& f) &&;  
template<class F> constexpr auto transform(F&& f) const &&;
```

[...]

-20- Constraints: is_move_constructible_v<E, decltype((error()))> is true.

[...]

[Issaquah 2023-02-09; Jonathan provides improved wording]

[Issaquah 2023-02-09; LWG]

Move to Immediate for C++23

Proposed resolution:

This wording is relative to [N4928](#).

1. Modify 22.8.6.7 [\[expected.object.monadic\]](#) as indicated:

```
template<class F> constexpr auto and_then(F&& f) &;  
template<class F> constexpr auto and_then(F&& f) const &;
```

[...]

-2- Constraints: is_ ~~copy~~ _constructible_v<E, decltype(error())> is true.

[...]

```
template<class F> constexpr auto and_then(F&& f) &&;  
template<class F> constexpr auto and_then(F&& f) const &&;
```

[...]

-6- Constraints: is_ ~~copy~~ _constructible_v<E, decltype(std::move(error()))> is true.

[...]

```
template<class F> constexpr auto or_else(F&& f) &;  
template<class F> constexpr auto or_else(F&& f) const &;
```

[...]

-10- Constraints: is_ ~~copy~~ _constructible_v<T, decltype(value())> is true.

[...]

```
template<class F> constexpr auto or_else(F&& f) &&;  
template<class F> constexpr auto or_else(F&& f) const &&;
```

[...]

-14- Constraints: is_ ~~copy~~ _constructible_v<T, decltype(std::move(value()))> is true.

[...]

```
template<class F> constexpr auto transform(F&& f) &;  
template<class F> constexpr auto transform(F&& f) const &;
```

[...]

-18- Constraints: is_ ~~copy~~ _constructible_v<E, decltype(error())> is true.

[...]

```
template<class F> constexpr auto transform(F&& f) &&;  
template<class F> constexpr auto transform(F&& f) const &&;
```

[...]

-22- Constraints: is_ ~~copy~~ _constructible_v<E, decltype(std::move(error()))> is true.


```
[...]

template<class F> constexpr auto transform_error(F&& f) &;
template<class F> constexpr auto transform_error(F&& f) const &;

[...]
```

-26- Constraints: is `_copy_constructible_v<T, decltype(value())>` is true.

```
[...]

template<class F> constexpr auto transform_error(F&& f) &&;
template<class F> constexpr auto transform_error(F&& f) const &&;

[...]
```

-30- Constraints: is `_copy_constructible_v<T, decltype(std::move(value()))>` is true.

2. Modify 22.8.7.7 [\[expected.void.monadic\]](#) as indicated:

```
template<class F> constexpr auto and_then(F&& f) &;
template<class F> constexpr auto and_then(F&& f) const &;

[...]
```

-2- Constraints: is `_copy_constructible_v<E, decltype(error())>` is true.

```
[...]

template<class F> constexpr auto and_then(F&& f) &&;
template<class F> constexpr auto and_then(F&& f) const &&;

[...]
```

-6- Constraints: is `_copy_constructible_v<E, decltype(std::move(error()))>` is true.

```
[...]

template<class F> constexpr auto transform(F&& f) &;
template<class F> constexpr auto transform(F&& f) const &;

[...]
```

-16- Constraints: is `_copy_constructible_v<E, decltype(error())>` is true.

```
[...]

template<class F> constexpr auto transform(F&& f) &&;
template<class F> constexpr auto transform(F&& f) const &&;

[...]
```

-20- Constraints: is `_copy_constructible_v<E, decltype(std::move(error()))>` is true.

```
[...]
```

[3878](#). import std; should guarantee initialization of standard iostreams objects

Section: 31.4.2 [\[iostream.objects.overview\]](#) **Status:** [Immediate](#) **Submitter:** Tim Song **Opened:** 2023-02-09 **Last modified:** 2023-02-10

Priority: Not Prioritized

Discussion:

In the old world, `#include <iostream>` behaves as if it defined a static-storage-duration `ios_base::Init` object, which causes the standard iostreams objects to be initialized (if necessary) on startup and flushed on shutdown.

But we don't include headers with `import std;`, so we need separate wording to provide this guarantee. The proposed resolution below was adapted from a suggestion by Mathias Stearn on the reflector.

[2023-02-09 Tim updates wording following LWG discussion]

[Issaquah 2023-02-09; LWG]

Move to Immediate for C++23

Proposed resolution:

This wording is relative to [N4928](#).

1. Modify 31.4.2 [\[iostream.objects.overview\]](#) p5 as indicated:

-5- The results of including `<iostream>` in a translation unit shall be as if `<iostream>` defined an instance of `ios_base::Init` with static storage duration. Each C++ library module (16.4.2.4 [\[std.modules\]](#)) in a hosted implementation shall behave as if it contains an interface unit that defines an unexported

[ios_base::Init](#) variable with ordered initialization (6.9.3.3 [\[basic.start.dynamic\]](#)).

[Note ?]: As a result, the definition of that variable is appearance-ordered before any declaration following the point of importation of a C++ library module. Whether such a definition exists is unobservable by a program that does not reference any of the standard iostream objects. — end note]

3879. `erase_if` for `flat_{,multi}set` is incorrectly specified

Section: 24.6.11.5 [\[flat.set.erasure\]](#), 24.6.12.5 [\[flat.multiset.erasure\]](#) **Status:** [Immediate](#) **Submitter:** Tim Song **Opened:** 2023-02-09 **Last modified:** 2023-02-10

Priority: Not Prioritized

Discussion:

The current specification of `erase_if` for `flat_{,multi}set` calls `ranges::remove_if` on the set, which is obviously incorrect — the set only present constant views of its elements.

[Issaquah 2023-02-09; LWG]

Move to Immediate for C++23

Proposed resolution:

This wording is relative to [N4928](#).

1. Modify 24.6.11.5 [\[flat.set.erasure\]](#) as indicated:

```
template<class Key, class Compare, class KeyContainer, class Predicate>
typename flat_set<Key, Compare, KeyContainer>::size_type
erase_if(flat_set<Key, Compare, KeyContainer>& c, Predicate pred);
```

~~-1- Effects: Equivalent to:~~

```
auto [erase_first, erase_last] = ranges::remove_if(c, pred);
auto n = erase_last - erase_first;
c.erase(erase_first, erase_last);
return n;
```

~~-1- Preconditions: Key meets the Cpp17MoveAssignable requirements.~~

~~-2- Effects: Let E be `bool(pred(as_const(e)))`. Erases all elements e in c for which E holds.~~

~~-3- Returns: The number of elements erased.~~

~~-4- Complexity: Exactly `c.size()` applications of the predicate.~~

~~-5- Remarks: Stable (16.4.6.8 [\[algorithm.stable\]](#)). If an invocation of `erase_if` exits via an exception, c is in a valid but unspecified state (3.66 [\[defns.valid\]](#)).~~

~~[Note 1: c still meets its invariants, but can be empty. — end note]~~

2. Modify 24.6.12.5 [\[flat.multiset.erasure\]](#) as indicated:

```
template<class Key, class Compare, class KeyContainer, class Predicate>
typename flat_multiset<Key, Compare, KeyContainer>::size_type
erase_if(flat_multiset<Key, Compare, KeyContainer>& c, Predicate pred);
```

~~-1- Effects: Equivalent to:~~

```
auto [erase_first, erase_last] = ranges::remove_if(c, pred);
auto n = erase_last - erase_first;
c.erase(erase_first, erase_last);
return n;
```

~~-1- Preconditions: Key meets the Cpp17MoveAssignable requirements.~~

~~-2- Effects: Let E be `bool(pred(as_const(e)))`. Erases all elements e in c for which E holds.~~

~~-3- Returns: The number of elements erased.~~

~~-4- Complexity: Exactly `c.size()` applications of the predicate.~~

~~-5- Remarks: Stable (16.4.6.8 [\[algorithm.stable\]](#)). If an invocation of `erase_if` exits via an exception, c is in a valid but unspecified state (3.66 [\[defns.valid\]](#)).~~

~~[Note 1: c still meets its invariants, but can be empty. — end note]~~

3880. Clarify operator+= complexity for `{chunk, stride}_view::iterator`

Section: 26.7.27.7 [\[range.chunk.fwd.iter\]](#), 26.7.30.3 [\[range.stride.iterator\]](#) **Status:** [Immediate](#) **Submitter:** Tim Song **Opened:** 2023-02-09 **Last modified:** 2023-02-10

Priority: Not Prioritized

Discussion:

The intent was that the precondition allows the call to `ranges::advance`, which otherwise would have time linear in the argument of `operator+=`, to actually be implemented using `operator+=` or equivalent for all but the last step. This is at best very non-obvious and should be clarified.

[Issaquah 2023-02-10; LWG issue processing]

Move to Immediate for C++23

Proposed resolution:

This wording is relative to [N4928](#).

1. Modify 26.7.27.7 [\[range.chunk.fwd.iter\]](#) p13 as indicated:

```
constexpr iterator& operator+=(difference_type x)
    requires random_access_range<Base>;
```

-12- *Preconditions*: If x is positive, `ranges::distance(current_, end_) > n_ * (x - 1)` is true.

[Note 1: If x is negative, the *Effects* paragraph implies a precondition. — end note]

-13- *Effects*: Equivalent to:

```
if (x > 0) {
    ranges::advance(current_, n_ * (x - 1));
    missing_ = ranges::advance(current_, n_ * x, end_);
} else if (x < 0) {
    ranges::advance(current_, n_ * x + missing_);
    missing_ = 0;
}
return *this;
```

2. Modify 26.7.30.3 [\[range.stride.iterator\]](#) p14 as indicated:

```
constexpr iterator& operator+=(difference_type n) requires random_access_range<Base>;
```

-13- *Preconditions*: If n is positive, `ranges::distance(current_, end_) > stride_ * (n - 1)` is true.

[Note 1: If n is negative, the *Effects* paragraph implies a precondition. — end note]

-14- *Effects*: Equivalent to:

```
if (n > 0) {
    ranges::advance(current_, stride_ * (n - 1));
    missing_ = ranges::advance(current_, stride_ * n, end_);
} else if (n < 0) {
    ranges::advance(current_, stride_ * n + missing_);
    missing_ = 0;
}
return *this;
```

3881. Incorrect formatting of container adapters backed by `std::string`

Section: 24.6.13 [\[container.adaptors.format\]](#) Status: [Immediate](#) Submitter: Victor Zverovich Opened: 2023-02-10 Last modified: 2023-02-10

Priority: Not Prioritized

Discussion:

According to 24.6.13 [\[container.adaptors.format\]](#) container adapters such as `std::stack` are formatted by forwarding to the underlying container:

```
template<class FormatContext>
    typename FormatContext::iterator
    format(maybe-const-adaptor& r, FormatContext& ctx) const;
```

Effects: Equivalent to: `return underlying_.format(r.c, ctx);`

This gives expected results for `std::stack<T>` and most types of underlying container:

```
auto s = std::format("{", std::stack{std::deque{'a', 'b', 'c'}});
// s == "[ 'a', 'b', 'c' ]"
```

However, when the underlying container is `std::string` the output is:

```
auto s = std::format("{", std::stack{std::string{"abc"}});
// s == "abc"
```

This is clearly incorrect because `std::stack` itself is not a string (it is only backed by a string) and inconsistent with formatting of ranges where non-string range types are formatted as comma-separated values delimited by '[' and ']'. The correct output in this case would be `[ 'a', 'b', 'c' ]`.

Here is an illustration of this issue on godbolt using `{fmt}` and an implementation of the formatter for container adapters based on the one from the standard: <https://godbolt.org/z/P1nrM1986>.

A simple fix is to wrap the underlying container in `std::views::all(_t)` (<https://godbolt.org/z/8MT1be838>).

Previous resolution [SUPERSEDED]:

This wording is relative to [N4928](#).

1. Modify 24.6.13 [\[container.adaptors.format\]](#) as indicated:

-1- For each of `queue`, `priority_queue`, and `stack`, the library provides the following formatter specialization where *adaptor-type* is the name of the template:

```
namespace std {
    template<class charT, class T, formattable<charT> Container, class... U>
    struct formatter<adaptor-type<T, Container, U...>, charT> {
    private:
        using maybe-const-adaptor =                // exposition only
            fmt-maybe-const<adaptor-type<T, Container, U...>, charT>;
        formatter<views::all t<const Container&>, charT> underlying_;           // exposition only

    public:
        template<class ParseContext>
        constexpr typename ParseContext::iterator
            parse(ParseContext& ctx);

        template<class FormatContext>
        typename FormatContext::iterator
            format(maybe-const-adaptor& r, FormatContext& ctx) const;
    };
}

[...]
```

```
template<class FormatContext>
    typename FormatContext::iterator
        format(maybe-const-adaptor& r, FormatContext& ctx) const;

-3- Effects: Equivalent to: return underlying_.format(views::all(r.c), ctx);
```

[2023-02-10 Tim provides updated wording]

The container elements may not be const-formattable so we cannot use the const formatter unconditionally. Also the current wording is broken because an adaptor is not range and we cannot use *fmt-maybe-const* on the adaptor — only the underlying container.

[Issaquah 2023-02-10; LWG issue processing]

Move to Immediate for C++23

Proposed resolution:

This wording is relative to [N4928](#).

1. Modify 24.6.13 [\[container.adaptors.format\]](#) as indicated:

-1- For each of `queue`, `priority_queue`, and `stack`, the library provides the following formatter specialization where *adaptor-type* is the name of the template:

```
namespace std {
    template<class charT, class T, formattable<charT> Container, class... U>
    struct formatter<adaptor-type<T, Container, U...>, charT> {
    private:
        using maybe-const-container =                // exposition only
            fmt-maybe-const<Container, charT>;
        using maybe-const-adaptor =                // exposition only
            fmt-maybe-const<is const v<maybe-const-container>, adaptor-type<T, Container, U...>, charT>; // see 26.2 [ranges.syn]

        formatter<ranges::ref view<maybe-const-container>Container, charT> underlying_;           // exposition only

    public:
        template<class ParseContext>
        constexpr typename ParseContext::iterator
            parse(ParseContext& ctx);

        template<class FormatContext>
        typename FormatContext::iterator
            format(maybe-const-adaptor& r, FormatContext& ctx) const;
    };
}

[...]
```

```
template<class FormatContext>
    typename FormatContext::iterator
        format(maybe-const-adaptor& r, FormatContext& ctx) const;

-3- Effects: Equivalent to: return underlying_.format(r.c, ctx);
```